MEDIAVAULT – DIGITAL MEDIA RIGHTS, LICENSING, DISTRIBUTION & ROYALTY MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

Pragadeesh S

240701388

in partial fulfilment of the End Semester Examination for the course

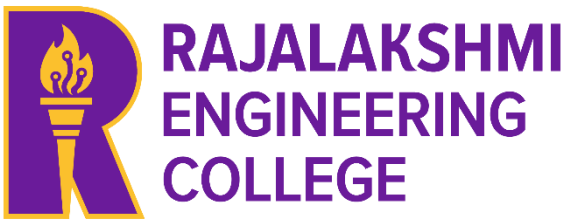
CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF ENGINEERING

COMPUTER SCIENCE ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

[OCTOBER-2026]

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "MediaVault – Digital Media Rights, Licensing, Distribution & Royalty Management System" is the bonafide work of Pragadeesh S (240701388), who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year [2026–2027], Semester [V], under my supervision.

SIGNATURE

Dr. J. MANORANJANI

HEAD OF THE DEPARTMENT

Department of CSE

Rajalakshmi Engineering College

Thandalam – 602105

SIGNATURE

Mr.BHUVANESWARAN B

COURSE FACULTY

Assistant Professor (SG)

Department of [CSE]

Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, Pragadeesh S (240701388), hereby declare that the project report titled “MediaVault – Digital Media Rights, Licensing, Distribution & Royalty Management System”, submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, pages, Deluge scripts, workflows and integrations — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

Place: [Chennai]
Date:

SIGNATURE OF THE STUDENT
Pragadeesh S
2116240701388

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman Thiru. S. Meganathan, Chairperson Dr. (Mrs.) Thangam Meganathan and Vice-Chairman Mr. Abhay Meganathan S for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, Dr. S. N. Murugesan, Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank Dr. J. Manoranjani, Head of the Department, Department of CSE, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, Mr.BHUVANESWARAN B

, Assistant Professor (SG), for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

Pragadeesh S

ABSTRACT

Digital media such as films and music is licensed, not sold: a rights holder grants a distributor the right to release a title in named territories for a fixed period. When licences are tracked in simple records, a distribution request can be approved although the licence has expired, covers another territory, or belongs to content whose ownership nobody has confirmed. Severity labels that users can edit, ownership typed in without verification and default permission profiles add to the risk.

MediaVault is a rights, licensing, distribution and royalty management application built on Zoho Creator by upgrading an existing prototype. Its data model links the Content, Rights_Holders, Content_Ownership, Licenses, Distribution_Requests, Rights_Violations, Revenue, Royalty_Rules and Royalty_Payments forms through lookup fields. When a distributor submits a request, a Deluge workflow compares it with the licences for that content by territory (or Worldwide), status and dates, sets the request status, and records one violation with a system-assigned severity (Critical, High, Medium or Low) and a reason. Ownership claims are saved as Pending and only the administrator can verify them, with verified shares capped at 100 per cent; a licence can be Active only when the ownership of its content is Verified. Three permission profiles (Content Owner, Distributor and Finance Manager), a role hierarchy and filtered reports separate access by role. A read-only Custom API protected by OAuth 2.0 publishes licence records with a computed coverage state, a Zia task writes a plain-English explanation of each violation, and a dashboard snippet shows key counts.

The application was tested role by role with separate logins and fictional sample data, and the defects found were fixed and logged. A request for Monsoon Letters in Australia with no covering licence produced a Critical violation, a distributor saw only its own requests once a filtered report replaced the all-records report, and unverified ownership blocked licence activation. Royalty automation was not built, and the project ran in a single environment without formal user acceptance testing.

Keywords: Low-Code Development; Zoho Creator; Deluge Scripting; Digital Rights Management; Licensing; Role-Based Access Control; Custom API; Zia

PROJECT AT A GLANCE

Attribute	Details
Project Title	MediaVault – Digital Media Rights, Licensing, Distribution & Royalty Management System
Application Domain	Media and Entertainment – digital rights, licensing, distribution and royalty management
Student Name & Register No.	Pragadeesh S – 240701388
Zoho Creator Edition	Professional (Enterprise Trial)
Application Name (Link Name)	MediaVault
Published / Live URL	https://creatorapp.zoho.in/pragadeesh.s.2024.cse_raj741/mediavault
Demo Credentials (per role)	Administrator: pragadeesh.s.2024.cse@rajalakshmi.edu.in Content Owner: pragadeeeee@gmail.com Distributor: pragadeeeee@gmail.com Finance Manager : pragadeeeee@gmail.com
Number of Forms / Subforms	9 forms; 1 legacy subform (Rights_Ownership in Content)
Number of Reports (by type)	List: 20 Kanban: 0 Calendar: 0 Pivot/Chart: 1 Others: 0
Number of Pages / Dashboards	1 dashboard (MediaVault Dashboard with KPI tile snippet)
Number of Workflows	Form workflows only (Table 5.3); Schedule: 17 Approval: 0 Blueprint: 0 Batch: 0 [7]
Deluge Functions / Custom Actions	10 scripts (Table 5.2), including 2 custom functions: Fetch_License_Records and explainViolation
Integrations	Zoho Creator Custom API (OAuth 2.0); Zoho Zia task (AI explanation of violations)
Roles & Profiles Defined	Administrator, Content Owner, Distributor, Finance Manager
Environments Used	Single application instance (development, stage and production environments not used)
Date of Production Deployment	02-NOV-2026

TABLE OF CONTENTS

ABSTRACT iv
PROJECT AT A GLANCE v
LIST OF TABLES ix
LIST OF FIGURES x
LIST OF ABBREVIATIONS xi
INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope of the Project 3
1.5 Justification for a Low-Code Approach 3
1.6 Organisation of the Report 3
LITERATURE SURVEY AND SYSTEM STUDY 5
2.1 Study of the Existing System 5
2.2 Limitations of the Existing System 6
2.3 Low-Code versus Traditional Development 6
2.4 Literature Survey 7
2.5 Emerging Trends in Low-Code Development 8
2.6 Proposed System 9
REQUIREMENT ANALYSIS 10
3.1 Stakeholders and User Roles 10
3.2 Functional Requirements 10
3.3 Non-Functional Requirements 12
3.4 Hardware and Software Requirements 12
3.5 Use Case Model 13
SYSTEM DESIGN 15
4.1 System Architecture 15
4.2 Data Model 15
4.3 Form Design 17
4.4 Process and Workflow Design 19
4.5 User Interface and Navigation Design 21
4.6 Access Control Design 21
IMPLEMENTATION 23
5.1 Application Setup 23
5.2 Forms and Data Capture 23

5.3 Reports 25
5.4 Pages and Dashboards 26
5.5 Deluge Scripting 27
5.6 Workflows and Automation 29
5.7 Integrations 30
5.8 User Management and Role-Based Access 33
5.9 AI and Analytics Features 34
5.10 Pro-Code Extension 35
5.11 Mobile and Portal Access 35
TESTING AND DEBUGGING 36
6.1 Testing Strategy 36
6.2 Test Cases 36
6.3 Debugging Deluge Scripts 38
6.4 Defect Log 39
6.5 User Acceptance Testing 40
DEPLOYMENT, SECURITY AND GOVERNANCE 41
7.1 Environments and Deployment Strategy 41
7.2 Performance Optimisation 41
7.3 Security Features 42
7.4 Governance and Administration 43
RESULTS AND DISCUSSION 44
8.1 Application Output 44
8.2 Achievement of Objectives 45
8.3 Comparison with the Existing System 46
8.4 Limitations of the Low-Code Approach 46
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum 46
CONCLUSION AND FUTURE ENHANCEMENTS 48
9.1 Conclusion 48
9.2 Future Enhancements 48
9.3 Learning Outcomes and Career Relevance 48
REFERENCES 49
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST 50
APPENDIX B – COURSE OUTCOME MAPPING 53
APPENDIX C – COMPLETE DELUGE SOURCE CODE 54
APPENDIX D – USER MANUAL 63
D.1 Accessing the Application 63
D.2 Instructions for the Administrator 63

D.3 Instructions for the Content Owner 63
D.4 Instructions for the Distributor 63
D.5 Instructions for the Finance Manager 63
APPENDIX E – EVALUATION RUBRIC 64
APPENDIX F – VIVA-VOCE RECORD SHEET 66

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application 7
Table 2.2: Summary of Literature Survey 8
Table 3.1: Stakeholders and User Roles 10
Table 3.2: Functional Requirements 10
Table 3.3: Non-Functional Requirements 12
Table 3.4: Hardware and Software Requirements 12
Table 3.5: Use Case Description – Submit Distribution Request and Automatic Rights Check 14
Table 4.1: Entities and Relationships 16
Table 4.2: Field Specification – Key MediaVault Forms 17
Table 4.3: Severity Classification Rules 20
Table 4.4: Role–Permission Matrix 22
Table 5.1: Reports Implemented 25
Table 5.2: Deluge Scripts Implemented 27
Table 5.3: Workflows Implemented 29
Table 5.4: Integrations Implemented 31
Table 6.1: Test Cases 36
Table 6.2: Defect Log 39
Table 7.1: Deployment Checklist 41
Table 7.2: Performance-Related Design Choices 42
Table 7.3: Security Controls Implemented 42
Table 8.1: Observed Output of the First Rights Check (REQ-003) 44
Table 8.2: Mapping of Objectives to Outcomes 45
Table 8.3: Existing System versus Proposed Application 46
Table A.1: Feature Coverage Checklist 50
Table B.1: Mapping of Course Outcomes to Report Chapters 53
Table E.1: Evaluation Rubric for the Individual Project 64
Table E.2: Performance Level Descriptors 65
Table F.1: Viva-Voce Questions and Responses 66
Table F.2: Marks Summary 66

LIST OF FIGURES

Figure 2.1: As-Is Process Flow of the Prototype Distribution Check 5
Figure 3.1: Use Case Diagram of MediaVault 13
Figure 4.1: System Architecture of MediaVault 15
Figure 4.2: Entity–Relationship Diagram of MediaVault 16
Figure 4.3: Workflow / Process Flow for the Distribution Rights Check 20
Figure 4.4: Navigation Structure of MediaVault by Role 21
Figure 5.1: Application Menu and the All Content Report with a Record Open..... 23
Figure 5.2: Content Form in Live Mode 24
Figure 5.3: Distribution Requests Form in Live Mode 24
Figure 5.4: Ownership Claim Saved as Pending (Content_Ownership Record) 24
Figure 5.5: My Requests Report as Seen by the Distributor..... 26
Figure 5.5a: MediaVault Dashboard: Active Licenses and Rights Violations Report 26
Figure 5.5b: MediaVault Dashboard: Revenue by Platform Chart, Royalty Payments and Distribution Requests 26
Figure 5.5c: MediaVault Dashboard: Tables and Six KPI Tiles 26
Figure 5.6: Form Workflows of Distribution_Requests 30
Figure 5.7: Form Workflows of Content_Ownership (Before the Third Workflow Was Added)..... 30
Figure 5.8: Custom API Builder – Response Step of Fetch_License_Records 31
Figure 5.9: Postman Call to Fetch_License_Records Without a Token (401, Code 1030) 32
Figure 5.10: Permission Profiles: Content Owner, Distributor and Finance Manager Added..... 33
Figure 5.11: Content Owner Permission Set..... 33
Figure 5.12: Finance Manager Permission Set (Revenue Edit Dialog Open) 34
Figure 5.13: Role Hierarchy: CEO above Content Owner, Distributor and Finance Manager 34

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
DS	Deluge Script
ER	Entity–Relationship
FR	Functional Requirement
HTML	HyperText Markup Language
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
LCAP	Low-Code Application Platform
NFR	Non-Functional Requirement
OAuth	Open Authorization
RBAC	Role-Based Access Control
REST	Representational State Transfer
UAT	User Acceptance Testing
UI	User Interface
URL	Uniform Resource Locator
WF	Workflow
Zia	Zoho's AI assistant

CHAPTER 1
INTRODUCTION

1.1 Background

Films, music, series and other digital media are rarely sold outright. They are licensed: a rights holder grants a distributor or streaming platform the right to distribute a title in named territories for a fixed period, and the revenue earned from that title is later shared among the rights holders in agreed percentages. Every distribution decision must therefore respect four properties of a licence: the title it covers, the territory, the start date and the end date.

When licences are tracked in simple records with one-line checks, it becomes easy to approve distribution that no licence covers. An expired licence can still look valid, a request outside the licensed territory can slip through, and royalties can be calculated against ownership shares that nobody has verified. Each of these is a rights violation of a different seriousness, and the organisation needs the system, not the person typing the record, to say which one it is.

MediaVault is a digital media rights, licensing, distribution and royalty management system built on the Zoho Creator low-code platform. It was developed in stages on top of an earlier Zoho Creator prototype: first the distribution check and the severity rules were corrected, then ownership verification was added, then role-based access, and finally a protected read-only API, an AI-written explanation of violations and a dashboard of key indicators.

1.2 Problem Statement

The prototype with which the project began could store content, licences, distribution requests, violations, revenue and royalties, but its rules were unsafe for real rights decisions:

- The distribution check approved a request whenever any Active licence existed for the same content and territory. It did not compare the licence start and end dates with the requested period, so an expired licence could approve a request.
- Every violation was saved with High severity whatever the cause, a separate workflow tried to guess severity from keywords typed in the Reason field, and

Severity and Request_Status were dropdowns that users could change themselves.

- The Licenses form had no mandatory fields, so Content, Territory and dates could be left blank, and no check stopped an end date earlier than the start date.
- Ownership was held in a subform inside Content with no verification. An owner could claim any percentage and nobody confirmed it.
- Only the default Read and Write permission profiles existed, so users could not be limited to the forms and records their role needs, and the Fetch_License_Records function returned every licence to any caller.

The problem addressed by this project is to build, on Zoho Creator, an application in which the system decides whether a request is covered by a licence, classifies the severity of every violation automatically, requires administrator verification of ownership, and shows each role only what it needs.

1.3 Objectives

- Validate every distribution request against licences by content, territory (or Worldwide), licence status and dates.
- Classify severity (Critical, High, Medium, Low) automatically from the validation result and stop users from setting severity or request status.
- Keep one violation record per request, with a reason that names the licence involved.
- Capture ownership claims as Pending, let only the administrator verify them, and cap verified shares at 100 per cent for each content item.
- Allow a licence to be saved as Active only when ownership of its content is Verified.
- Separate access for the Content Owner, Distributor and Finance Manager roles using permission profiles, a role hierarchy and filtered reports, with no Delete rights.
- Expose licence status to outside systems through a protected read-only Custom API, add an AI-written explanation to violations, and give the administrator a dashboard of key indicators.

1.4 Scope of the Project

In scope: content and rights-holder records; ownership claims with administrator verification; licences with mandatory fields and an ownership check before activation; distribution requests with an automatic rights check and severity classification; violation records with a Zia-written explanation; revenue, royalty rule and royalty payment records with entry-time validation; permission profiles, roles and filtered reports for three non-administrator roles; the Fetch_License_Records Custom API; and an administrator dashboard with KPI tiles.

Out of scope: automatic generation of royalty payments from revenue and ownership shares (planned as a later stage and not built); matching a licence to one specific distributor; per-role dashboards; owner-specific royalty views; e-mail notifications; a separate front end or back end outside Zoho Creator; and deployment to a separate production environment.

1.5 Justification for a Low-Code Approach

The first plan for MediaVault considered a React front end with a Python FastAPI back end and a SQLite database. The project was moved to Zoho Creator because the prototype already existed there and because the platform supplies forms, reports, permission profiles, roles, workflows and the Deluge scripting language without separate hosting. Forms create the tables and entry screens automatically, so the effort could be spent on the rules that matter: licence matching, severity classification and ownership verification.

Those rules are written once in Deluge and run on the server every time a form is submitted, so they do not depend on what a user's browser sends. The trade-offs of this choice, such as the limited visibility of what the platform does with permissions and the fact that an exported application carries structure but not data, are discussed in Chapters 6 and 8.

1.6 Organisation of the Report

Chapter 2 reviews the existing prototype and the related low-code sources and states the proposed system. Chapter 3 lists the requirements and the main use case. Chapter 4 presents the architecture, data model, process and access design. Chapter 5 documents the implementation. Chapter 6 reports testing, defects and fixes. Chapter 7 covers deployment, security and governance as far as they apply to this project. Chapter 8

discusses the results and Chapter 9 concludes and lists future work. The appendices contain the feature checklist, course-outcome mapping, source code, user manual, evaluation rubric and viva-voce record sheet.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

The existing system is the Zoho Creator prototype with which this project began. It had eight forms (Content, Rights Holders, Licenses, Distribution Requests, Revenue, Royalty Rules, Royalty Payments and Rights Violations), reports for active licences, revenue, royalty payments, violations and distribution requests, a dashboard with a Revenue by Platform chart, workflows for distribution validation, royalty calculation, licence-expiry warning and risk classification, and a Custom API named Fetch_License_Records that had been tested in Postman.

Its distribution validation fetched the licences of the requested content and approved the request if any licence had status Active and the same territory as the request. Otherwise it set the status to Rights Violation and inserted a violation with severity High. Figure 2.1 shows this flow and the weaknesses it carried.

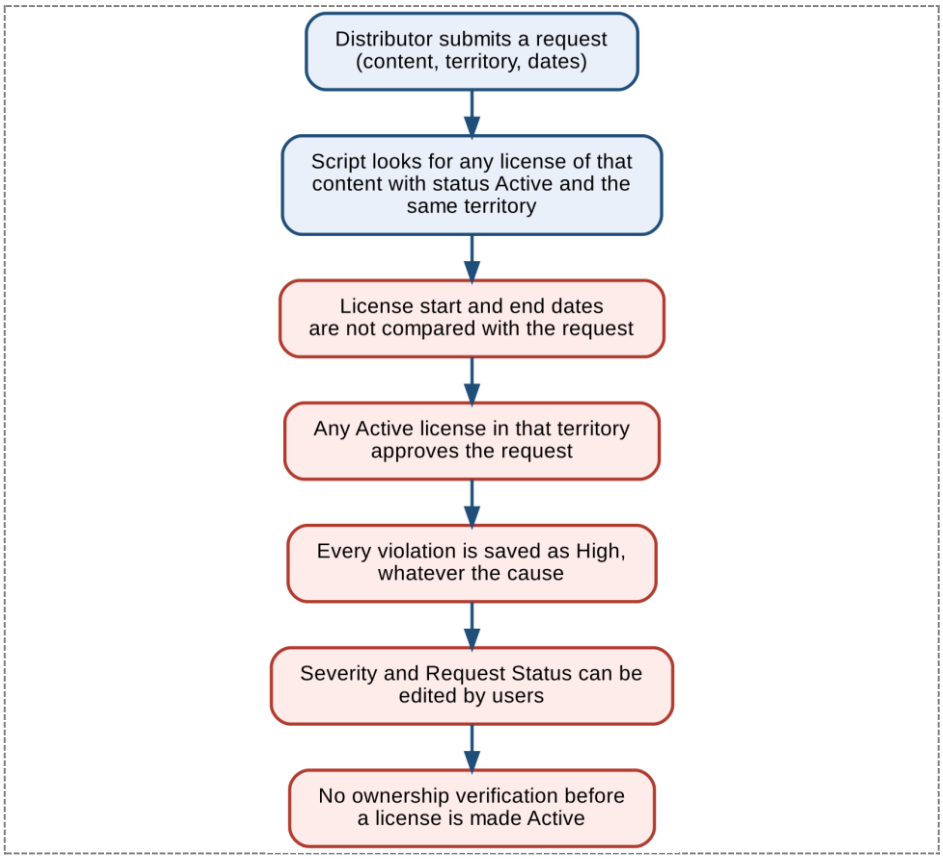


Figure 2.1: As-Is Process Flow of the Prototype Distribution Check

2.2 Limitations of the Existing System

- Licence start and end dates were never compared with the requested period, and the distributor was not matched to the licensee.
- Every violation was saved as High, and a separate risk-classification workflow chose severity from keywords in the Reason text and could overwrite a correct value on edit.
- Severity and Request_Status could be edited by users, and Request_Status was mandatory on the request form.
- Licences could be saved without content, territory or dates, and a request with its end date before its start date could pass the check.
- Ownership percentages were typed into a subform and never verified.
- The licence-expiry workflow overwrote the licence notes on every save, and royalty payments were entered by hand with a typed percentage and uncontrolled status changes.
- Fetch_License_Records returned every licence to anyone holding the key.

2.3 Low-Code versus Traditional Development

Table 2.1 compares the traditional route first considered for this project with the low-code route actually taken. The comparison is qualitative; no effort or performance measurements were taken.

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Infrastructure	Server, database and hosting must be set up; the first plan used React, FastAPI and SQLite.	Hosted by Zoho; no installation. The application runs in the India data centre.
Data model and screens	Tables, migrations, API endpoints and forms are coded separately.	Each form creates its table and entry screen; lookups link forms.
Business rules	Written in the back-end language and tested with automated tests.	Written in Deluge as form workflows and functions; tested manually with scenarios.
Access control	Login, password hashing and role checks are built by the developer.	Permission profiles, a role hierarchy and filtered reports are configured in Settings.
Integration	REST endpoints are coded and secured by the developer.	A Deluge function is exposed as a Custom API with OAuth 2.0.
AI features	An external model API and its key must be integrated.	The Zia task is called from Deluge; no separate key was needed.
Portability	Source code can be moved freely.	The .ds export restores structure only; records, invited users and API settings are not carried over.
Control and visibility	Full control of every behaviour.	Some behaviour (for example how record visibility follows roles) had to be discovered by testing.

2.4 Literature Survey

Table 2.2 lists the sources consulted. Only sources that could be verified are included; summaries are limited to what the sources themselves state.

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	A. Bock and U. Frank [1], 2021	Low-Code Platform	Discussion paper on the low-code platform concept; its keywords include citizen developer, organisational agility and software development productivity.	Frames MediaVault as a build in which the platform supplies data, interface and workflow.
2	A. Sahay, A. Indamutsa, D. Di Ruscio and A. Pierantonio [2], 2020	Supporting the understanding and comparison of low-code development platforms	Technical survey of eight low-code platforms using a conceptual comparison framework; platforms are heterogeneous and hard to evaluate.	Supports checking a platform's features against requirements before choosing it.
3	Zoho Corporation [3]	Creating and managing custom APIs (Zoho Creator Help)	A Custom API exposes a Deluge function; only super admins and admins can create it; authentication is OAuth 2.0 (default) or a public key; responses are Standard or Custom.	Basis of Fetch_License_Records, built with OAuth 2.0 and a Standard response.
4	Zoho Corporation [4]	Zia task (Zoho Deluge Help)	The Zia task uses large language models from Deluge; syntax response = Zia[message: ...]; documented limits include a 40-second timeout.	Basis of the explainViolation function that writes the violation explanation.

2.5 Emerging Trends in Low-Code Development

Three themes in the sources shaped the design. First, low-code platforms are positioned for citizen developers and for organisational agility [1], which matches a one-student project that has to deliver working rules quickly. Second, platforms differ widely [2], so the choice of Zoho Creator was checked against concrete needs: forms with lookups, profiles and roles, scripting, a custom API and an AI task. Third, AI is now available

inside the scripting language [4]; because the same request must always receive the same answer, the project uses it only to explain a decision that fixed rules have already made.

2.6 Proposed System

The proposed system is the Zoho Creator application MediaVault. Content owners register content and submit ownership claims, which stay Pending until the administrator verifies them. The administrator creates licences, and a licence can be Active only when the ownership of its content is Verified. Distributors submit distribution requests; a Deluge workflow checks each request against the licences, sets the request status, and creates one violation with a system-assigned severity and a reason whenever the request cannot be approved. A Zia task adds a plain-English explanation to the violation. Finance managers record revenue and royalty payments. Profiles, roles and filtered reports keep each role to its own work, and a read-only Custom API and a dashboard serve outside systems and the administrator.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Administrator	The account owner, at the top (CEO) of the role hierarchy.	Verify ownership claims, create and activate licences, review violations, manage users, profiles and roles, and view the dashboard.
Content Owner	A rights holder who registers content. Zoho profile and role: Content Owner.	Add content, rights-holder records and ownership claims; cannot verify, edit or delete claims.
Distributor	A distribution or streaming platform. Zoho profile and role: Distributor.	Submit distribution requests and view own requests through the My Requests report; cannot change request status.
Finance Manager	Staff who handle money. Zoho profile and role: Finance Manager.	Add and edit revenue and royalty payment records; view content and royalty rules read-only.

3.2 Functional Requirements

The Status column records what the project actually delivered; items marked Not built are listed so that the report does not overstate the result.

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Status
FR-01	Register content with ID, title, type, language, release year, rights status and description.	Content Owner, Administrator	Implemented
FR-02	Submit an ownership claim (content, rights holder, percentage); the claim is saved as Pending and the submitter cannot verify it.	Content Owner	Implemented
FR-03	Administrator verifies claims; Verified By and Verification Date are recorded; verified shares for one content item cannot exceed 100%; the content's Ownership Status becomes Pending, Partially Verified, Verified or Rejected.	Administrator	Implemented
FR-04	Allow one non-rejected claim per rights holder per content item, with a percentage between 0.01 and 100.	Content Owner, Administrator	Implemented
FR-05	Licence records require content, licensee, territory, start date, end date and status.	Administrator	Implemented

FR ID	Requirement	Role(s)	Status
FR-06	Reject a licence or request whose end date is before its start date.	Administrator, Distributor	Implemented for requests; licence check not confirmed
FR-07	A licence can be saved as Active only if the Ownership Status of its content is Verified.	Administrator	Implemented
FR-08	Check every distribution request against licences by content, territory (or Worldwide), status Active and dates, and set the request status.	Distributor	Implemented
FR-09	Record which licence approved a request (Approved License lookup).	Distributor	Not built
FR-10	Assign severity automatically (Critical, High, Medium, Low); users cannot edit Severity or Request Status.	All	Implemented
FR-11	Keep one violation per request, with a reason naming the licence where one exists.	Administrator	Implemented
FR-12	Add a Zia-written plain-English explanation to each violation, labelled as AI-generated and never used for decisions.	Administrator	Implemented
FR-13	Limit menus and records by role using profiles, the role hierarchy and filtered reports.	All	Implemented for Distributor and Finance Manager; Content Owner "My" reports not confirmed
FR-14	Validate royalty entry: royalty = revenue x percentage / 100 rounded to 2 decimals, total percentage per revenue not above 100, no duplicate holder, Paid records locked.	Finance Manager	Implemented
FR-15	Expose licences, with a Coverage_State computed from dates, through a read-only Custom API protected by OAuth 2.0.	Administrator, external systems	Implemented
FR-16	Show KPI tiles (open violations, critical violations, pending claims, active licences, licences expiring in 30 days, pending requests) on the dashboard.	Administrator	Implemented
FR-17	Generate royalty payments automatically from revenue and verified ownership shares.	Finance Manager	Not built
FR-18	Match a licence to one specific distributor.	Distributor	Not built

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Security	Each role shall open only the forms and reports in its permission profile; no role shall have Delete rights.
NFR-02	Data integrity	Severity and request status shall be set by the system; saved values shall be restored if a user edits them.
NFR-03	Determinism	The same request shall always receive the same decision; AI text shall never change a decision.
NFR-04	Privacy	The AI prompt shall contain only the facts needed (content, territory, dates, severity, reason) and no e-mail addresses or phone numbers.
NFR-05	Usability	Forms shall use dropdowns, lookups and mandatory fields so that users select rather than type values.
NFR-06	Maintainability	Tunable switches (for example createAdvisoryRecords) shall be declared at the top of the script.
NFR-07	Portability	The application structure shall be recoverable from a .ds export; records are re-imported from CSV files.
NFR-08	Auditability	Each violation shall carry its reason, and verified claims shall carry the verifier and date.

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator, India data centre (creatorapp.zoho.in); trial edition
Scripting	Deluge (form workflows, custom functions, page HTML snippet)
Client device	Any computer with a current web browser and internet access; Google Chrome was used for development and testing
API testing	Postman
API credentials	Zoho API Console (api-console.zoho.in), Self Client, scope ZohoCreator.customapi.EXECUTE
AI	Zia task in Deluge; no separate API key
Test data	Seven CSV files of fictional data (rights holders, content, licences, requests, revenue, royalty rules, royalty payments)
Test users	Additional Zoho users invited through Share with the Content Owner, Distributor and Finance Manager profiles, used in a separate browser window

3.5 Use Case Model

Figure 3.1 shows the actors and use cases. The automatic licence check is included in every distribution request, and the AI explanation is included in the check.

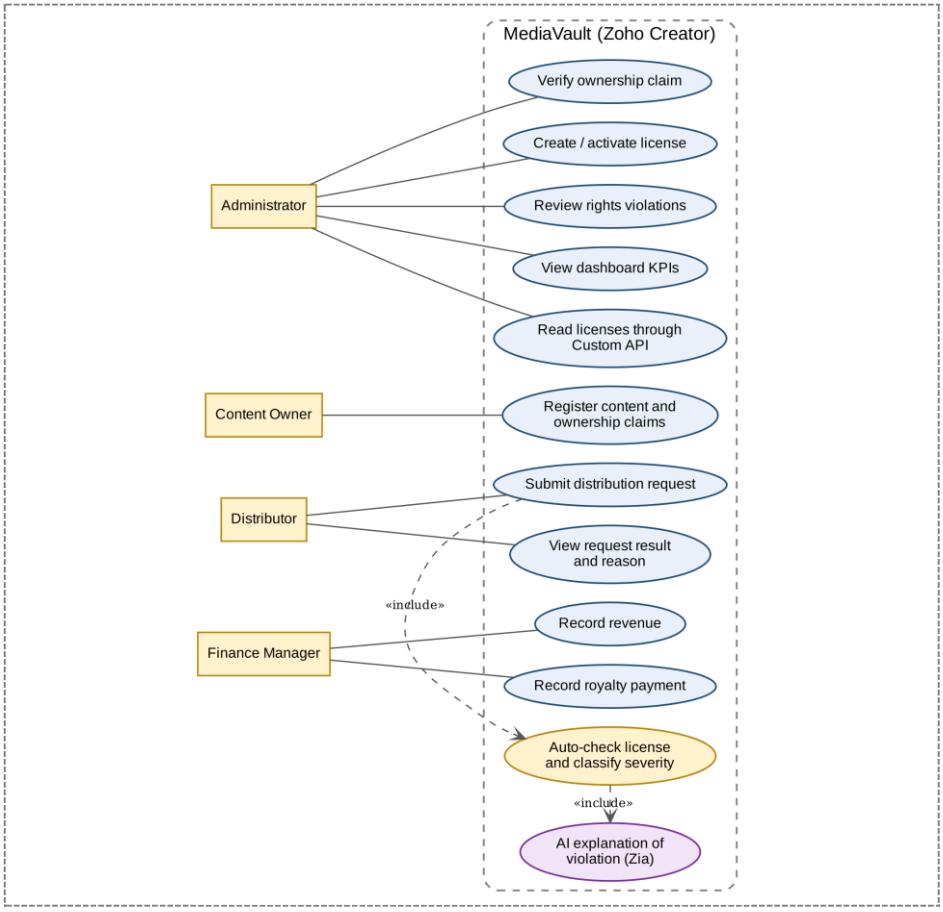


Figure 3.1: Use Case Diagram of MediaVault

Table 3.5: Use Case Description – Submit Distribution Request and Automatic Rights Check

Use Case ID	UC-01
Use Case Name	Submit Distribution Request and Automatic Rights Check
Actors	Distributor (primary); Administrator (reviews violations)
Pre-conditions	Content and licences exist; the distributor is signed in with the Distributor profile.
Main Flow	1. The distributor opens Distribution Requests and enters request ID, content, platform, territory, dates and expected revenue. 2. On validation, the end date is checked against the start date. 3. On success, the rights check fetches the licences of the content for the territory or Worldwide. 4. If an Active licence covers every requested day, Request Status becomes Approved. 5. Otherwise Request Status becomes Rights Violation and a violation is created with system-assigned severity and reason. 6. explainViolation adds the Zia explanation to the violation.
Alternate Flows	End date before start date: the save is blocked with an alert. No licence for the content and territory: severity Critical. Licence exists but is not Active, is expired or does not cover the dates: severity High. Approved, but the licence ends within 30 days of the requested end: Medium advisory.
Post-conditions	The request has a status set by the system; at most one violation exists for the request.
Exceptions	If the Zia call fails, the explanation field shows an unavailable message and the decision is unaffected.

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

MediaVault has four layers (Figure 4.1). Users reach it through a browser under one of four identities. The Zoho Creator application holds the forms, the Deluge workflows and functions that apply the rules, and the reports and dashboard. Records are stored in the Zoho Creator cloud database, where lookup fields link forms and profiles and roles control visibility. At the edge, CSV import loads sample data, the Custom API exposes licences, and the Zia task writes explanations.

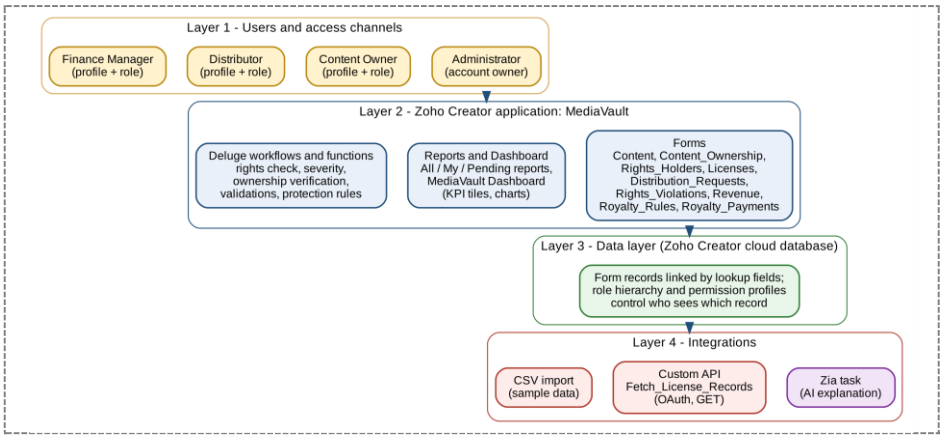


Figure 4.1: System Architecture of MediaVault

4.2 Data Model

The data model has nine forms linked by lookup fields (Figure 4.2). Content_Ownership is the join between Content and Rights_Holders and carries the percentage and verification status. Licenses, Distribution_Requests and Revenue refer to Content, Rights_Violations refers to the request that caused it, and Royalty_Rules and Royalty_Payments connect licences, revenue and rights holders. Table 4.1 lists the entities and relationships.

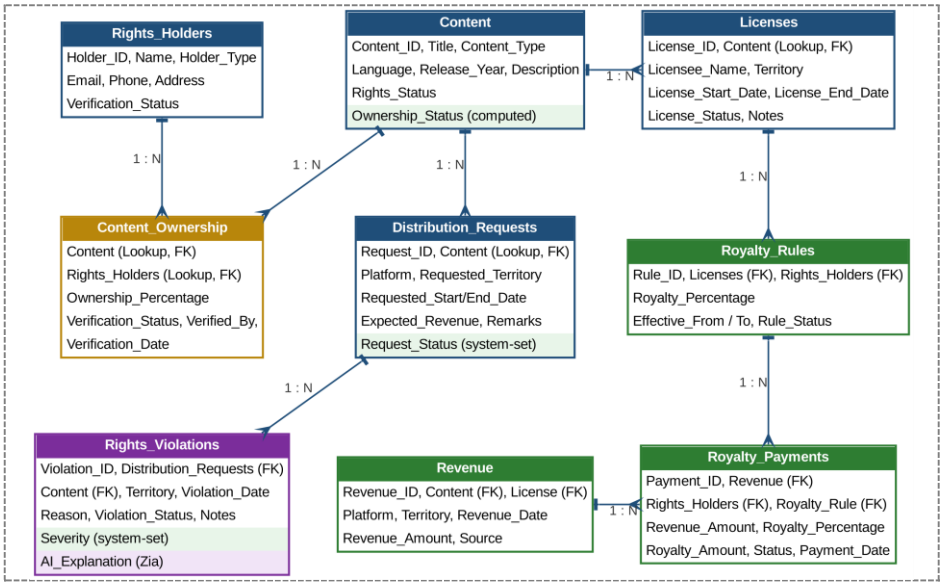


Figure 4.2: Entity–Relationship Diagram of MediaVault

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Rights_Holders	Holder_ID, Name, Holder_Type, Email	Content_Ownership; Royalty_Rules; Royalty_Payments	One-to-Many	Lookups in the child forms
Content	Content_ID, Title, Content_Type, Language, Rights_Status, Ownership_Status	Content_Ownership; Licenses; Distribution_Requests; Revenue	One-to-Many	Content lookup in each child form
Content_Ownership	Ownership_Percentage, Verification_Status, Verified_By, Verification_Date	Content; Rights_Holders	Many-to-One (join)	Two lookups
Licenses	License_ID, Licensee_Name, Territory, License_Start_Date, License_End_Date, License_Status	Content; Royalty_Rules; Revenue	One-to-Many	Content lookup; Licenses lookup in rules
Distribution_Requests	Request_ID, Platform, Requested_Territory, dates, Expected_Revenue, Request_Status	Content; Rights_Violations	One-to-Many	Content lookup; request lookup in violations

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Rights_Violations	Violation_ID, Territory, Reason, Severity, Violation_Status, AI_Explanation	Distribution_Requests; Content	Many-to-One	Two lookups
Revenue	Revenue_ID, Platform, Territory, Revenue_Date, Revenue_Amount	Content; Licenses; Royalty_Payments	One-to-Many	Lookups
Royalty_Rules and Royalty_Payments	Royalty_Percentage, Rule_Status; Royalty_Amount, Status, Payment_Date	Licenses, Revenue, Rights_Holders	Many-to-One	Lookups

4.3 Form Design

Table 4.2 specifies the fields of the forms whose layout was inspected during the project. Field types for the remaining forms were created in the prototype and are not repeated here.

Table 4.2: Field Specification – Key MediaVault Forms

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Form: Content	—	—	—	—	—
Content ID, Title	Content_ID, Title	Single line	Yes	—	—
Content Type, Language, Rights Status	Content_Type, Language, Rights_Status	Dropdown	Type and Rights Status	—	—
Release Year, Description	Release_Year, Description	Number; multi line	No	—	—
Ownership Status	Ownership_Status	Dropdown	No	Pending, Partially Verified, Verified, Rejected; set by workflow	Not editable by users
Form: Content_Ownership	—	—	—	—	—
Content	Content	Lookup (Content)	Yes	—	—

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Rights Holders	Rights_Holders	Lookup (Rights_Holders)	Yes	One non-rejected claim per holder per content	Link name is plural
Ownership Percentage	Ownership_Percentage	Decimal	Yes	0.01 to 100	—
Verification Status	Verification_Status	Dropdown	No	Pending (default), Verified, Rejected	Disabled for non-administrators
Supporting Document	Supporting_Document	File upload	No	—	—
Verified By, Verification Date	Verified_By, Verification_Date	Single line; date	No	Filled by workflow	Hidden on load
Form: Distribution_Requests	—	—	—	—	—
Request ID, Platform	Request_ID, Platform	Single line	Yes	—	—
Content	Content	Lookup (Content)	Yes	—	—
Requested Territory	Requested_Territory	Dropdown	Yes	—	—
Requested Start / End Date	Requested_Start_Date, Requested_End_Date	Date	Yes	End not before start	On Validate workflow
Expected Revenue, Remarks	Expected_Revenue, Remarks	Decimal; multi line	No	—	Remarks written by the check
Request Status	Request_Status	Dropdown	No	Set by the rights check	Hidden from users
Form: Licenses	—	—	—	—	—
Content, Licensee Name, Territory, Start, End, Status	Content, Licensee_Name, Territory, License_Start_Date, License_End_Date, License_Status	Lookup, text, dropdown, dates, dropdown	Yes	Status values include Active, Pending, Expired, Suspended	Active needs Verified ownership
Form: Rights_Violations	—	—	—	—	—
Severity	Severity	Dropdown	No	Critical, High, Medium, Low; set by workflow	Not editable by users

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Violation Status	Violation_Status	Dropdown	No	Open, Under Review, Resolved	—
AI Explanation	AI_Explanation	Multi line	No	Written by explainViolation	Labelled AI-generated

4.4 Process and Workflow Design

Figure 4.3 shows how a distribution request is processed. After the date check, the workflow gathers the licences of the content for the requested territory or Worldwide. If one is Active and covers every requested day, the request is Approved, with a Medium advisory when that licence ends within 30 days of the requested end. If none covers the request, severity is Critical when no licence exists for the content and territory, and High when a licence exists but is not Active, is expired or does not cover the dates. The violation is then inserted, or updated if one already exists, and the explanation is written.

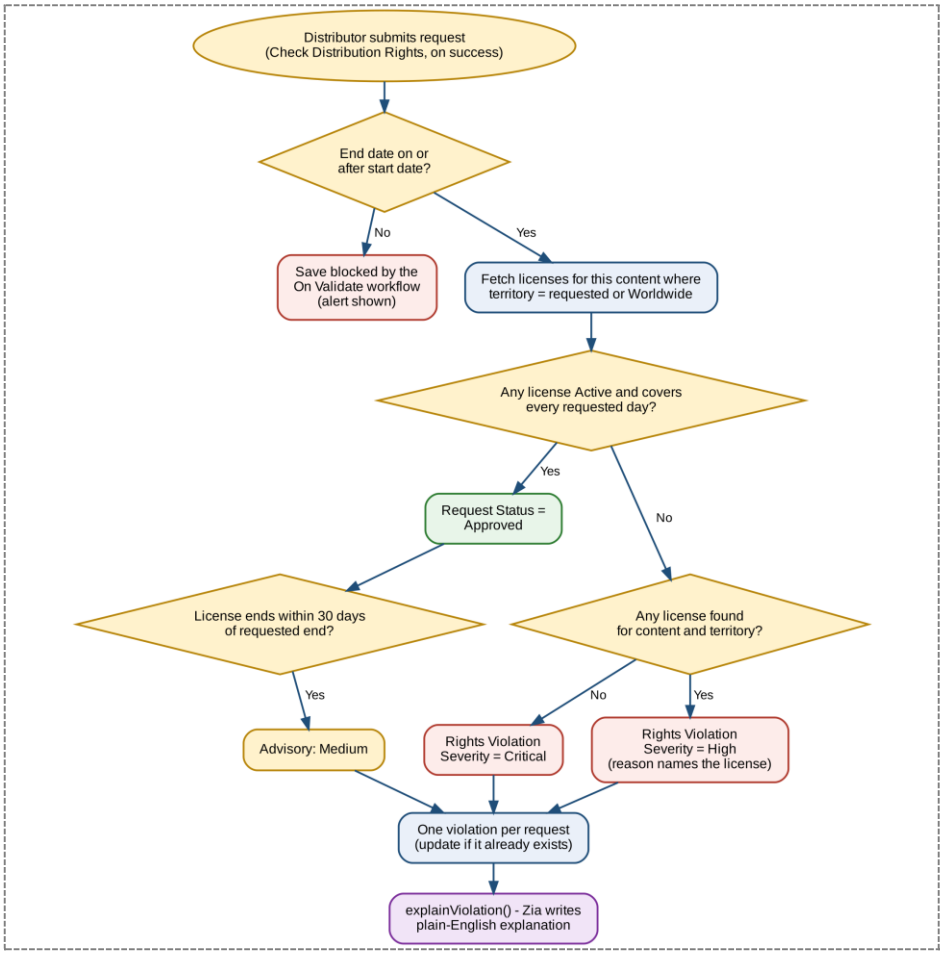


Figure 4.3: Workflow / Process Flow for the Distribution Rights Check

The severity rules are project-defined, not legal classifications. Table 4.3 summarises them; Low is the approved-with-administrative-gap case.

Table 4.3: Severity Classification Rules

Severity	Condition	Request outcome
Critical	No licence exists for the content and territory (or Worldwide).	Rights Violation
High	A licence exists but is not Active, is expired, or does not cover the requested dates; the reason names the licence.	Rights Violation
Medium	An Active licence covers the request but ends within 30 days of the requested end date.	Approved, with advisory
Low	The request is approved but an administrative gap exists (deployed script: royalty percentage or expected revenue missing).	Approved, with advisory

4.5 User Interface and Navigation Design

Each role sees a different menu (Figure 4.4). The administrator sees everything, including the MediaVault Dashboard. The Content Owner sees the Content, Content_Ownership and Rights Holders forms. The Distributor sees the Distribution Requests form and the My Requests report. The Finance Manager sees Revenue and Royalty Payments for entry and Content and Royalty Rules as read-only reports.

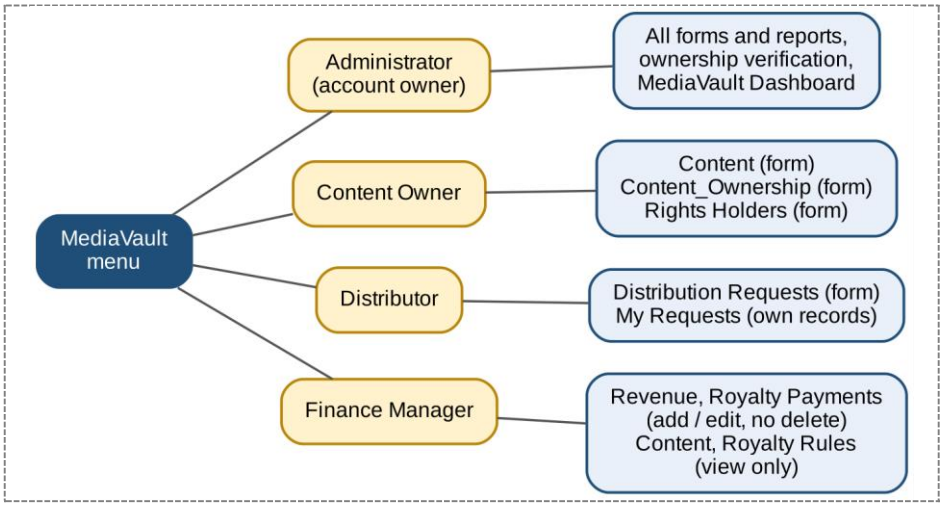


Figure 4.4: Navigation Structure of MediaVault by Role

4.6 Access Control Design

Access combines three mechanisms. Permission profiles decide which components a role can open and whether it can add, view, edit or delete. The role hierarchy (CEO above Content Owner, Distributor and Finance Manager, with sharing among peers switched off) decides whose records a user sees. Because testing showed that the hierarchy alone did not hide other users' sample records from a Distributor, filtered "My" reports (Added User equals the logged-in user) were used and the all-records report was removed from that profile. Ownership verification is restricted to the account owner in the script itself. Table 4.4 shows the resulting matrix.

Table 4.4: Role–Permission Matrix

Component	Administrator	Content Owner	Distributor	Finance Manager
Content	A E V D	A V	—	V (All Content)
Content_Ownership	A E V D (verify)	A V	—	—
Rights Holders	A E V D	A V	—	—
Licenses	A E V D	V (reports)	—	Not confirmed
Distribution Requests	A E V D	—	A; V (My Requests)	—
Rights Violations	A E V D	V (reports)	—	—
Revenue	A E V D	—	—	A E V
Royalty Rules	A E V D	—	—	V
Royalty Payments	A E V D	V (reports)	—	A E V
MediaVault Dashboard	Yes	No	No	No

Key: A = Add (form access), E = Edit, V = View (reports), D = Delete, — = no access. Delete is not granted to any non-administrator profile. Licenses, Rights Violations and Royalty Payments report access for the Content Owner was left as set during configuration and was not separately tested.

CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

MediaVault runs in the Zoho Creator India data centre. The project upgraded the existing application in place rather than rebuilding it. When the trial period lapsed, the application was re-imported from its .ds export, which restored forms, reports, workflows and pages but not records, invited users or API settings; sample records were therefore loaded again from CSV files and the Custom API was created again in the new account. The left menu lists the forms Content, Rights Holders, Licenses, Distribution Requests, Revenues, Royalty Rules, Royalty Payments and Rights Violations, with Content_Ownership added during Stage 2 (Figure 5.1).

The sample data are fictional: three content titles (Monsoon Letters, Neon Streets and Kaveri Nights), rights holders, six licences, seven distribution requests, three revenue records, five royalty rules and five royalty payments. The seven request records had to be entered through the form, because no violations appeared after the CSV import and the rights check runs when a request is submitted through the form.

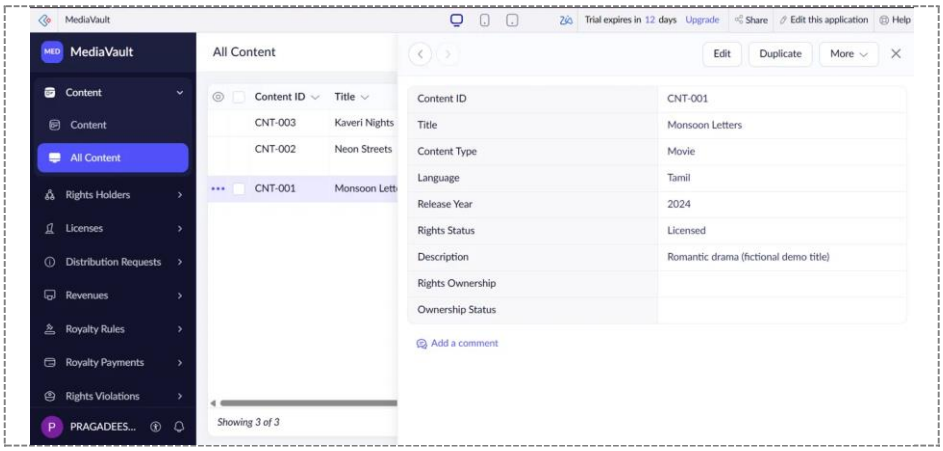


Figure 5.1: Application Menu and the All Content Report with a Record Open

5.2 Forms and Data Capture

The Content form (Figure 5.2) captures the content ID, title, type, language, release year, rights status and description, and holds the Ownership Status that the system maintains. The Distribution Requests form (Figure 5.3) captures the request ID, content, platform, territory, requested dates, expected revenue and remarks; Request Status is hidden from the user because the system sets it.

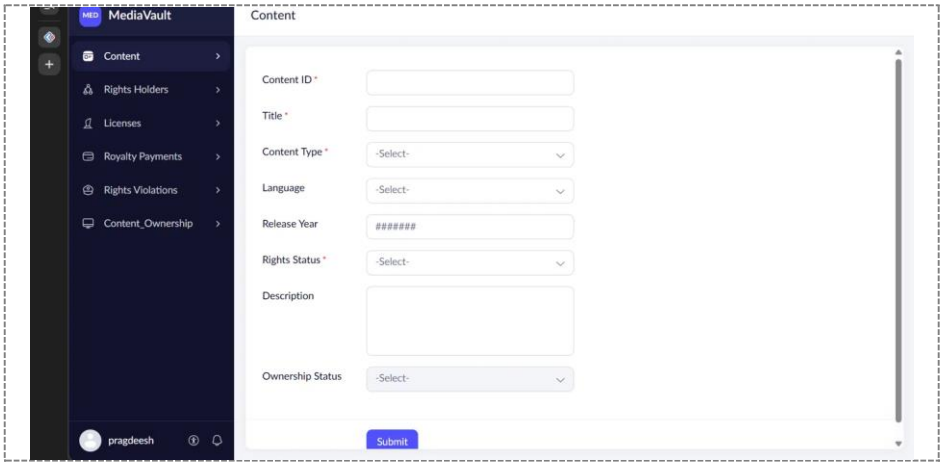


Figure 5.2: Content Form in Live Mode

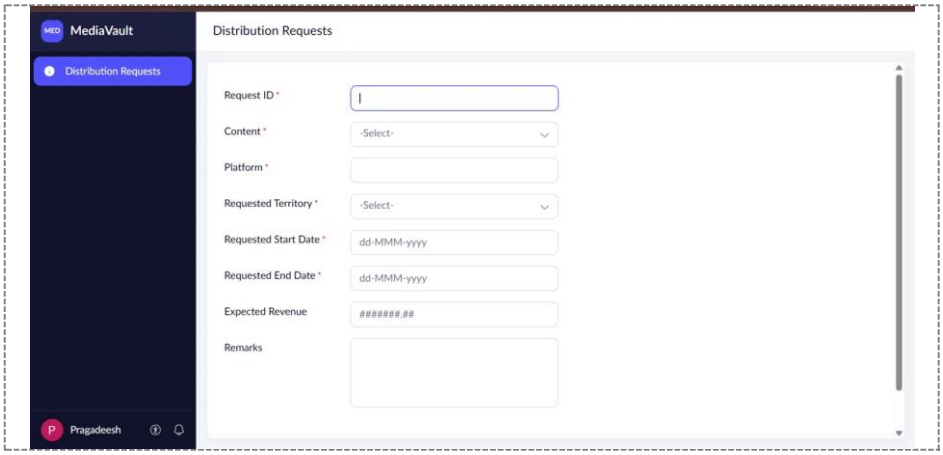


Figure 5.3: Distribution Requests Form in Live Mode

Ownership is captured in the separate Content_Ownership form, which replaced the unverified subform inside Content. A new claim is always saved as Pending, with Verified By and Verification Date blank (Figure 5.4).



Figure 5.4: Ownership Claim Saved as Pending (Content_Ownership Record)

5.3 Reports

Reports are list reports on the forms. Table 5.1 lists the reports confirmed during the project. The original application had ten reports; the three marked "added" were created during this project, and the exact final count should be read from the application.

Table 5.1: Reports Implemented

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
All Content	List	Content	None recorded	None recorded	Per profile (Table 4.4); Finance view-only
All Ownership (added)	List	Content_Ownership	None recorded	None recorded	Per profile (Table 4.4)
Pending Ownership Claims (added)	List	Content_Ownership	Verification Status equals Pending	None recorded	Administrator
My Requests (added)	List	Distribution_Requests	Added User equals logged-in user	None recorded	Distributor
Distribution Requests Report	List	Distribution_Requests	None recorded	None recorded	Per profile (Table 4.4)
Active Licenses	List	Licenses	Status Active (date filter not confirmed)	None recorded	Per profile (Table 4.4)
Rights Violations Report	List	Rights_Violations	None recorded	None recorded	Per profile (Table 4.4)
All Revenues, Revenue Report	List	Revenue	None recorded	None recorded	Finance Manager
All Royalty Payments, Royalty Payments Report	List	Royalty_Payments	None recorded	None recorded	Finance Manager
All Royalty Rules	List	Royalty_Rules	None recorded	None recorded	Finance view-only

The My Requests report is the key access-control report: after it replaced the all-records report in the Distributor profile, a Distributor saw only its own request (Figure 5.5).



Figure 5.5: My Requests Report as Seen by the Distributor

5.4 Pages and Dashboards

The MediaVault Dashboard page holds a title, four stacked report tables and a Revenue by Platform bar chart (Figures 5.5a and 5.5b). An HTML snippet was added that counts records directly from the forms and shows six KPI tiles (Figure 5.5c): open violations, critical open violations, pending ownership claims, active licences, licences expiring within 30 days and pending requests (Code C.10 in Appendix C). The first version failed with the error "Reached end of the page action" because the opening block ended with }%> instead of %>; the corrected snippet rendered correctly. In the final dashboard the tiles read 8, 3, 0, 3, 1 and 0 respectively. The dashboard is visible to the administrator only, because per-role dashboards were not built.

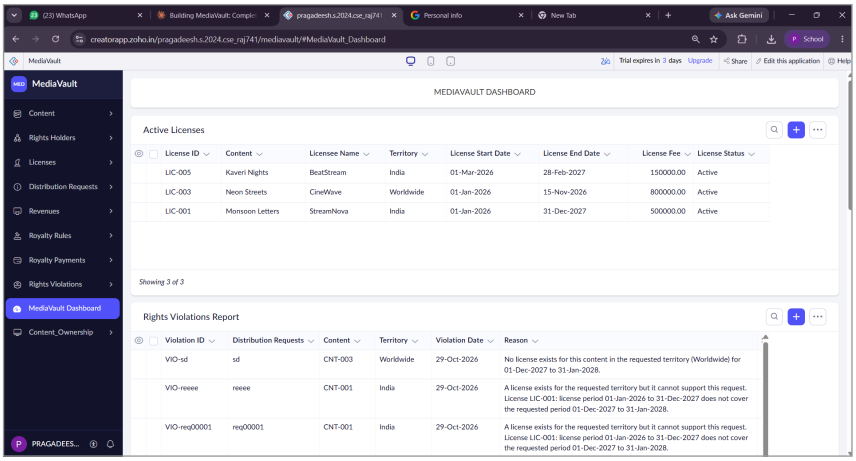


Figure 5.5a: MediaVault Dashboard: Active Licenses and Rights Violations Report

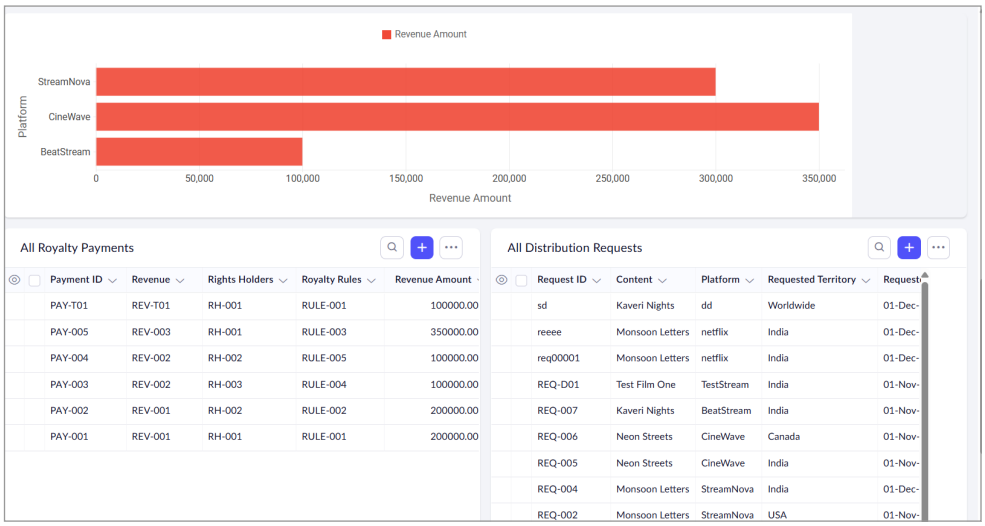


Figure 5.5b: MediaVault Dashboard: Revenue by Platform Chart, Royalty Payments and Distribution Requests

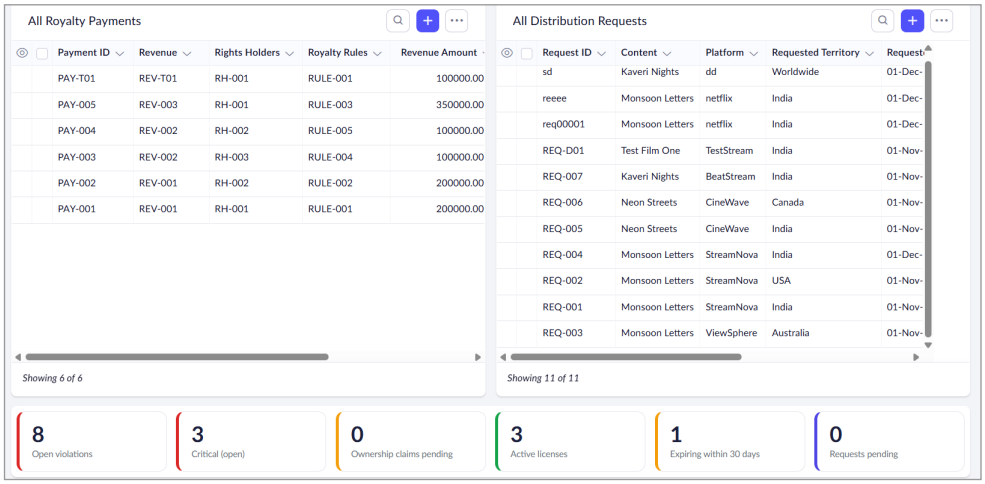


Figure 5.5c: MediaVault Dashboard: Tables and Six KPI Tiles

5.5 Deluge Scripting

Table 5.2 lists the scripts in the application. Complete listings are in Appendix C. The scripts follow one principle: the system derives status, severity and ownership state from stored data, and rejects or overwrites any value a user tries to supply.

Table 5.2: Deluge Scripts Implemented

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-01	Distribution_Requests → Check Distribution Rights (On Success)	Match the request to licences, set Request Status, create or update the violation with severity and reason	fetch with criteria, for each, if / else, insert into, date functions
DS-02	Check Distribution Rights (end of script)	Call explainViolation for the request's violations	for each, function call

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-03	Content_Ownership → On Load	Hide verification fields; disable status for non-administrators	hide, disable, zoho.loginuser
DS-04	Content_Ownership → On Validate	Range check, one claim per holder, 100% cap, lock of verified rows, verifier and date	fetch, loops, alert, cancel submit
DS-05	Content_Ownership → Update Content Ownership Status (On Success)	Recompute the Ownership Status of the content	loops, accumulation, update record
DS-06	Licenses → On Validate	Block Active licences for content whose ownership is not Verified	fetch, alert, cancel submit
DS-07	Distribution_Requests → On Validate	Reject end date before start date	comparison, alert, cancel submit
DS-08	Function Fetch_License_Records	Return licences with a computed Coverage_State for the Custom API	Map, List, date functions
DS-09	Function explainViolation	Ask Zia for a plain-English explanation and store it	Zia task, try / catch
DS-10	Dashboard page → HTML snippet	Count records for the KPI tiles	count(), page markup

The most important guard is the activation check on licences (Code 5.1). It reads one value, the Ownership Status of the content record, and its message was observed during testing when a licence was edited before ownership had been verified.

Code 5.1: DS-06 – Licence activation guard

```
// Script ID : DS-06
// Location  : Licenses -> Validations on form submission
if(input.License_Status == "Active")
{
    c = Content[ID == input.Content];
    if(c.Ownership_Status != "Verified")
    {
        alert "Cannot activate this license: ownership of this content
is
            not verified yet.";
        cancel submit;
    }
}
```

5.6 Workflows and Automation

All workflows are form workflows; no schedule, approval, Blueprint or batch workflow was used, because the design avoids scheduled workflows and none of the rules needs manager approval. Table 5.3 lists the workflows seen in the application; the workflow lists for Licenses, Rights_Violations and Royalty_Payments were not captured and are not itemised.

Table 5.3: Workflows Implemented

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Hide Request Status	Form	Distribution_Requests – On Load	Add and edit	Hide Request Status from the user	—
Check Distribution Rights	Form	Distribution_Requests – On Success	On add	Rights check, severity, violation, explanation call	DS-01, DS-02
Protect Request Status	Form	Distribution_Requests – On Validate (edit)	On edit	Restore the saved status	—
Date validation 2	Form	Distribution_Requests – On Validate	Add and edit	Block end date before start date	DS-07
hideVerified_By	Form	Content_Ownership – On Load	Add and edit	Hide fields; lock status for non-administrators	DS-03
3b. Workflow	Form	Content_Ownership – On Validate	Add and edit	Validate and stamp verification	DS-04
Update Content Ownership Status	Form	Content_Ownership – On Success	Add and edit	Recompute Content Ownership Status	DS-05
Licence activation guard	Form	Licenses – On Validate	Add and edit	Block Active without Verified ownership	DS-06

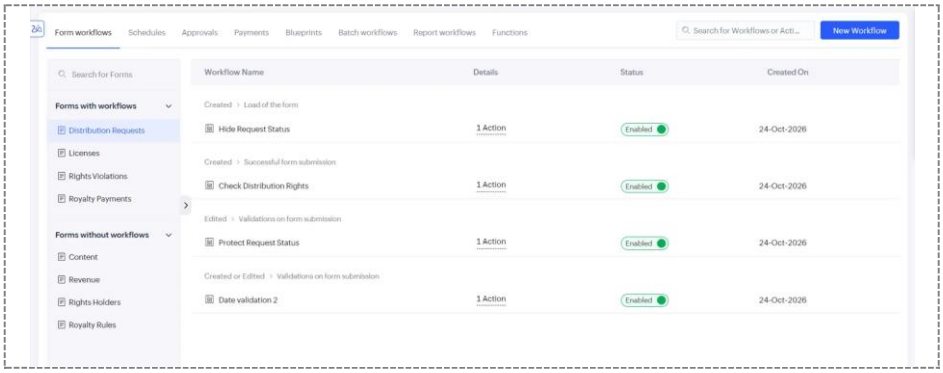


Figure 5.6: Form Workflows of Distribution_Requests

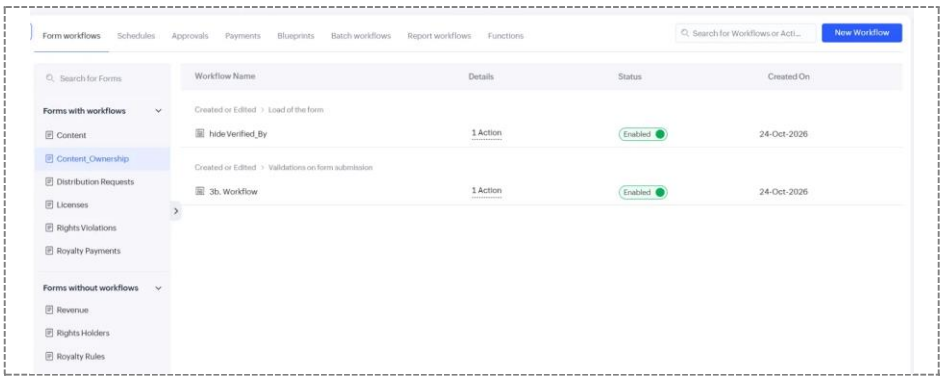


Figure 5.7: Form Workflows of Content_Ownership (Before the Third Workflow Was Added)

5.7 Integrations

MediaVault has one outbound interface and one AI task (Table 5.4). The Custom API Fetch_License_Records exposes the Deluge function DS-08 as a read-only GET endpoint. It is created in the Microservices area of the Creator home page, not inside the application, and only admins can create it [3]. It uses OAuth 2.0, with a Self Client created in the Zoho API Console and the scope ZohoCreator.customapi.EXECUTE, and the Standard response type, which wraps the function result in Creator's own code and result structure.

Table 5.4: Integrations Implemented

Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
Licence read API	Custom API Fetch_License_Records (Zoho Creator Microservices)	GET, no parameters, Standard response	OAuth 2.0, Self Client, scope ZohoCreator.customapi.EXECUTE, Zoho-oauthtoken header	Returns every licence with Coverage_State
AI explanation	Zia task inside Deluge	response = Zia[message : prompt]	None needed; runs inside Zoho	Facts of one violation in, plain-English explanation out
Sample data load	CSV import	Report menu → Import Data	Signed-in user	Seven CSV files of fictional data

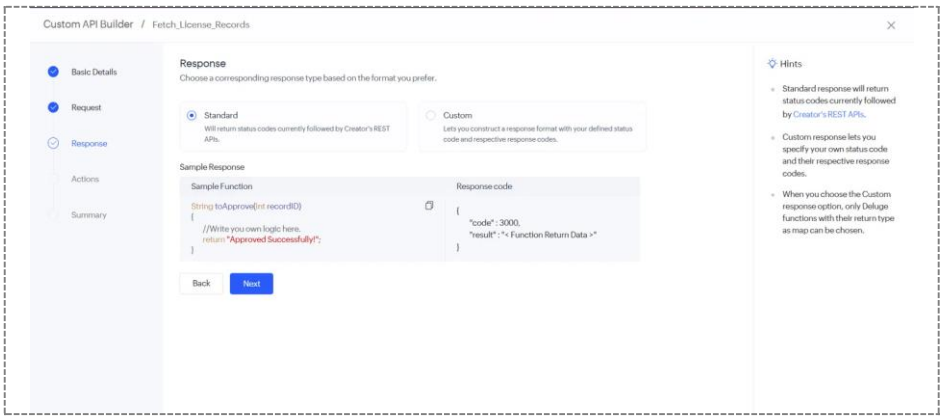


Figure 5.8: Custom API Builder – Response Step of Fetch_License_Records

Calling the endpoint from Postman without a token returned HTTP 401 with code 1030, "Authorization Failure. The access token is either invalid or has expired", which shows that the endpoint exists and refuses unauthenticated callers (Figure 5.9). A token was then generated through the Self Client and the call was reported working; the response body was not captured, so the report does not reproduce it.

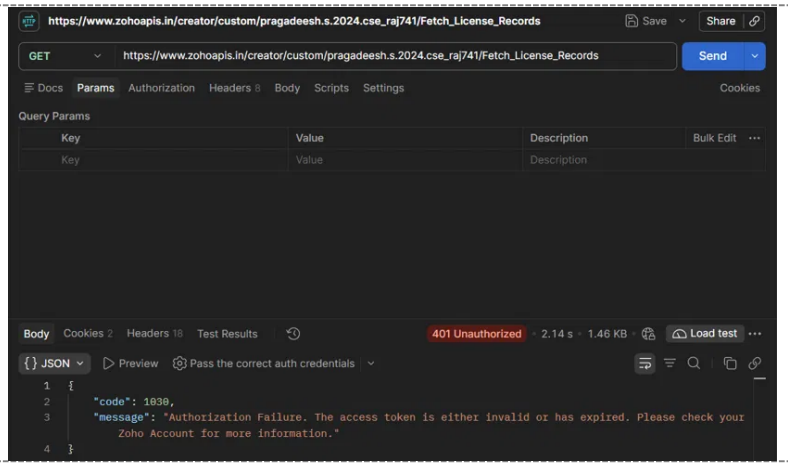


Figure 5.9: Postman Call to Fetch_License_Records Without a Token (401, Code 1030)

The function computes a Coverage_State from the dates on every call: Valid, Expiring Soon (ends within 30 days), Expired (Active but past its end date), Not Started, or Not Active with the stored status in brackets. An outside system therefore never sees a stale "Active" label (excerpt in Code 5.2).

Code 5.2: DS-08 – Coverage_State logic in Fetch_License_Records (excerpt)

```
if(lic.License_Status == "Active")
{
    if(lic.License_End_Date != null && lic.License_End_Date <
todayDate)
    {
        state = "Expired";
    }
    else if(lic.License_Start_Date != null && lic.License_Start_Date >
        todayDate)
    {
        state = "Not Started";
    }
    else if(lic.License_End_Date != null && lic.License_End_Date <=
        todayDate.addDay(30))
    {
        state = "Expiring Soon";
    }
    else
    {
        state = "Valid";
    }
}
```

Limits of the API: it only reads; anyone holding a valid token sees every licence because it ignores the Distributor, Owner and Finance roles; the token lasts about an hour; and the API setting is not part of the .ds export.

5.8 User Management and Role-Based Access

Three permission profiles were added to the default Read and Write profiles (Figure 5.10). Each profile ticks Access only for the forms its role needs, leaves Edit and Delete unticked except for Finance entry, and has dashboard access switched off. The Content Owner profile is shown in Figure 5.11 and the Finance Manager profile in Figure 5.12.

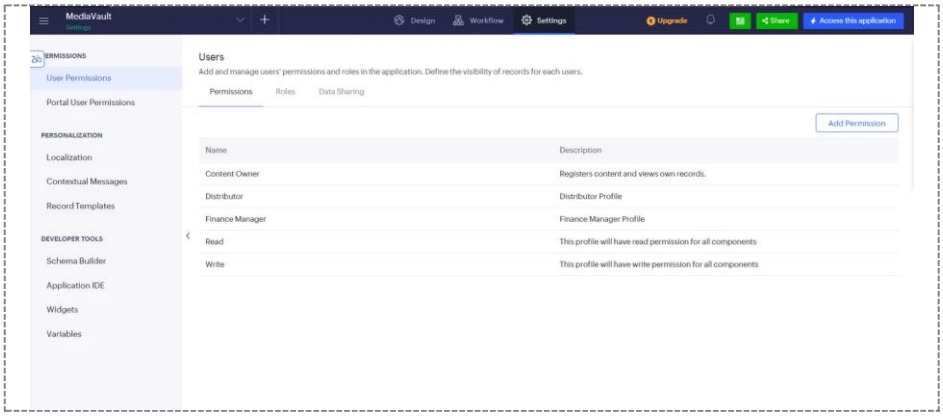


Figure 5.10: Permission Profiles: Content Owner, Distributor and Finance Manager Added

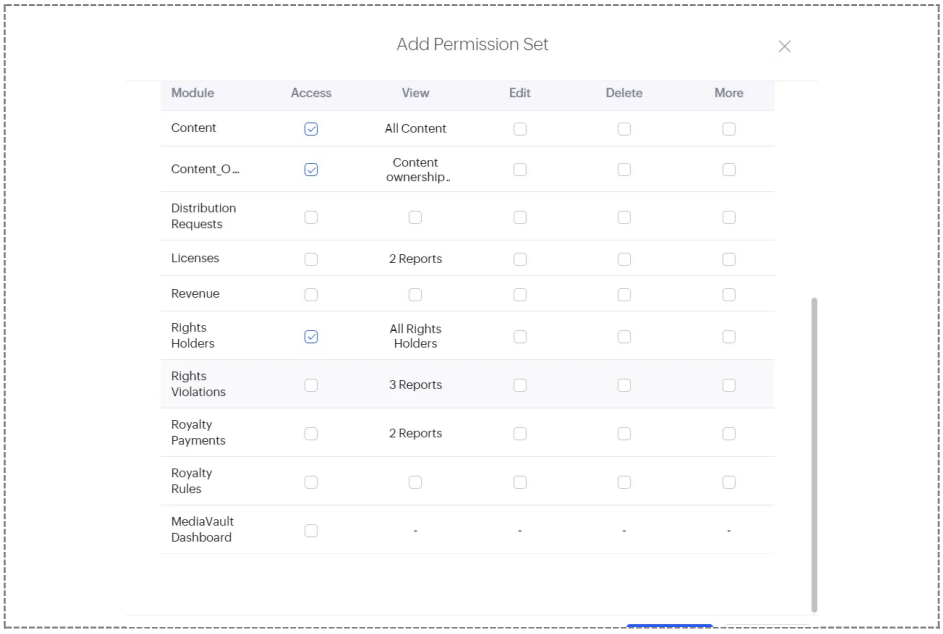


Figure 5.11: Content Owner Permission Set

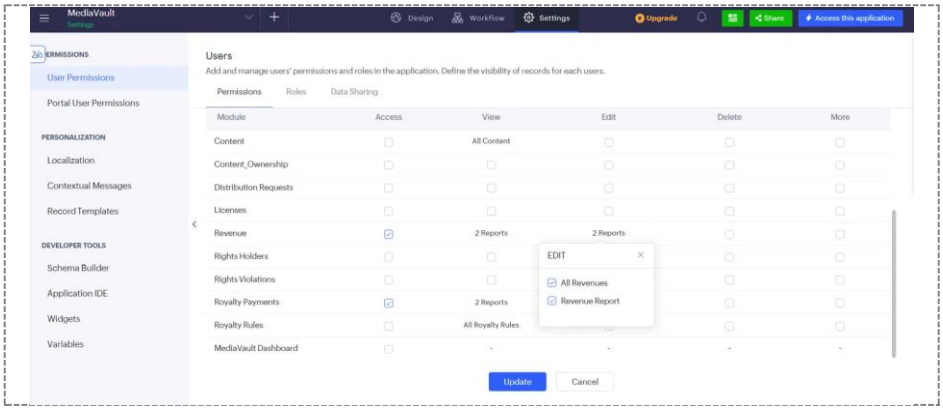


Figure 5.12: Finance Manager Permission Set (Revenue Edit Dialog Open)

Roles form a hierarchy with the administrator account at the top as CEO and the three business roles beneath it (Figure 5.13). Share Data with Peers is switched off so that two owners or two distributors cannot see each other's records. A second Zoho user had to be added to the account before it could be invited to the application, because the first invitation failed with a message that the e-mail was not a user in the account.

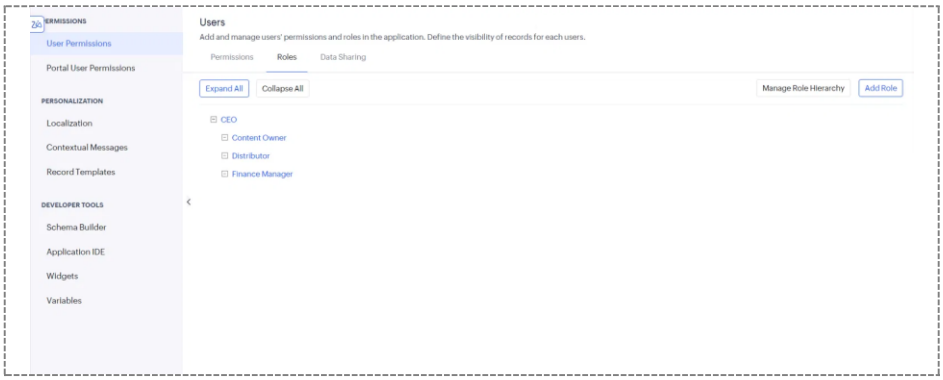


Figure 5.13: Role Hierarchy: CEO above Content Owner, Distributor and Finance Manager

5.9 AI and Analytics Features

AI is used for one purpose: to explain a decision the rules have already made. The explainViolation function sends only the facts of the violation (content title, territory, requested period, severity and the system reason) to the Zia task and stores the answer in AI_Explanation, prefixed with a notice that the stored Reason and Severity are the official result. The function is called at the end of Check Distribution Rights, because a first attempt that triggered it from a workflow on Rights_Violations left the field blank for violations inserted by a script.

Code 5.3: DS-09 – Zia call in explainViolation (excerpt)

```
try
{
    response = Zia
    [
        message : prompt
    ];
    v.AI Explanation = "AI-generated summary. The stored Reason and
        Severity are the official result. " + response.toString();
}
catch (e)
{
    v.AI_Explanation = "AI explanation unavailable: " + e;
}
```

The first prompt let the model add advice that was not in the facts. For the request Monsoon Letters, India, 01-Dec-2027 to 31-Jan-2028, the explanation spoke of an "overlap issue" and told the distributor to contact "the licensing authority immediately"; neither exists in the system. The prompt was tightened to allow only three next steps (ask the rights owner to extend or renew the licence, request a licence for the missing territory, or resubmit with dates inside the licensed period). The next output called the distributor a "licensor" and used the word "broadcast", which the facts never mentioned, so a further prompt was proposed; the student decided to leave the wording as it was, because the explanation is labelled AI-generated and never changes severity or status. Analytics consist of the KPI tiles described in Section 5.4.

5.10 Pro-Code Extension

No pro-code extension was used. There is no Catalyst function, widget or external service in MediaVault; all logic is Deluge inside Zoho Creator. The first plan for a React and FastAPI application was dropped before any code was written (Section 1.5).

5.11 Mobile and Portal Access

The Creator mobile app and customer portal were not used. All role testing was done in the web application, with the administrator in one browser window and the test user signed in separately in a private window.

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Testing was manual and scenario based, using fictional data. Three kinds of test were run: rule tests, in which requests and ownership claims were entered through the forms and the saved values compared with the expected result; role tests, in which a second Zoho user was signed in with each profile and the menu, visible records and permitted actions were checked; and negative tests, in which a blocked save or a refused call was the expected result. No automated tests were written and no external users took part.

6.2 Test Cases

Table 6.1 lists the tests whose outcome was reported or observed during the project. Where only the tester's confirmation exists, the status says so. The remaining Stage 1 request scenarios (REQ-001, REQ-002, REQ-004 to REQ-007) were prepared with expected results, but their individual outcomes were not recorded, so they are not claimed here.

Table 6.1: Test Cases

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-08, FR-10	Request with no licence in the territory	REQ-003: Monsoon Letters, ViewSphere, Australia, 01-Nov-2026 to 31-Dec-2026	Rights Violation; severity Critical set by the system	Violation created: Critical, Open, reason "No license exists for this content in the requested territory (Australia)…"	Pass
TC-02	FR-02	Ownership claim starts as Pending	Add Monsoon Letters, Meridian Pictures, 60.00	Saved as Pending; Verified By and Date blank	Saved as Pending; fields blank (Figure 5.4)	Pass
TC-03	FR-04	Duplicate claim blocked	Add the same content and holder again	Blocked with message	"This rights holder already has an ownership claim for this content."	Pass

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-04	FR-07	Activate a licence before ownership is verified	Edit LIC-001 with status Active	Blocked with message	Message shown; cause was a missing status-update workflow, then fixed (D-02)	Pass after fix
TC-05	FR-03, FR-07	Verify claims to 100% and save the licence	Verify all rows of a content item; save the licence	Content Verified; licence saves	Tester reported all Stage 2 checks passed	Pass (reported)
TC-06	FR-13	Content Owner role	Sign in as owner; add own content and claim	Limited menu; status locked	Tester reported "all good"	Pass (reported)
TC-07	FR-13, FR-10	Distributor menu, lookup and request	Sign in as distributor; open menu; submit a request	Only request form; Status hidden; Content list works; violation created	All three checks passed; request produced Rights Violation, Critical	Pass
TC-08	FR-13	Distributor sees only own records	Open the menu report	Only own request	First saw all sample requests (D-03); after My Requests, only REQ-D01 (Figure 5.5)	Pass after fix
TC-09	FR-13	Finance menu and add rights	Sign in as finance manager	Revenue and Royalty Payments for entry; others hidden or view-only	Content and Royalty Rules opened as add forms (D-04); fixed to view-only	Pass after fix
TC-10	FR-14	Finance entry rules	Add revenue REV-T01 (LIC-001, 100000); royalty payment; duplicate; Paid; Delete; direct URL to Distribution_Requests	Saved; duplicate blocked; Paid locked; no Delete; URL denied	Tester reported "all good"	Pass (reported)

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-11	FR-15	Custom API without a token	GET the endpoint in Postman	401 refusal	HTTP 401, code 1030 (Figure 5.9)	Pass
TC-12	FR-15	Custom API with an OAuth token	GET with Zoho-oauthtoken header	code 3000 and licence list	Tester reported it working; body not captured	Pass (reported)
TC-13	FR-12	Zia explanation	Request Monsoon Letters, India, 01-Dec-2027 to 31-Jan-2028	AI_Explanation filled	Blank at first (D-06); then plain text beginning "AI-generated summary…"	Pass after fix
TC-14	FR-16	KPI tiles	Paste the HTML snippet on the dashboard	Six tiles show counts	Error at first (D-07); tiles reported OK after correction	Pass after fix

6.3 Debugging Deluge Scripts

Zoho reports a Deluge error with a line number, which was pasted back and analysed. The clearest case was the ownership validation: the script used the link name Rights_Holder, and saving failed with "Error at line number: 32 Variable Rights_Holder does not exist in Content_Ownership". Zoho names the link name of a lookup to Rights_Holders in the plural, as it had already done in Royalty_Payments. The fix was to use the real link name (Code 6.1); the lesson, applied afterwards, was to read each field's link name in its properties before writing a script that uses it.

Code 6.1: DS-04 – Link-name fix in the one-claim-per-holder check

```
// before (failed): Rights_Holder == input.Rights_Holder
// after:
for each  p in Content_Ownership[Content == input.Content &&
    Rights_Holders == input.Rights_Holders && Verification_Status !=
    "Rejected"]
{
    if(input.ID == null || p.ID != input.ID)
    {
        errorMsg = "This rights holder already has an ownership claim
for
            this content.";
    }
}
```

6.4 Defect Log

Table 6.2: Defect Log

Bug ID	Description	Root Cause	Fix Applied	Status
D-01	Ownership save failed: Variable Rights_Holder does not exist.	Link name of the lookup is Rights_Holders.	Used the real link name in DS-04.	Closed
D-02	Licence could not be activated although ownership was 100% verified.	Content Ownership Status was blank; the status-update workflow did not exist.	Added the Update Content Ownership Status workflow (DS-05).	Closed
D-03	Distributor saw all sample requests.	The role hierarchy did not limit visibility as assumed.	Created My Requests (Added User = logged-in user); removed the all-records report from the profile.	Closed
D-04	Finance Manager could add Royalty Rules and Content.	Access was ticked, which gives the form.	Unticked Access; left View on All Content and All Royalty Rules.	Closed
D-05	CSV import produced no violations.	Import did not run the rights check.	Requests were entered through the form.	Closed
D-06	AI Explanation field blank.	Workflow on Rights_Violations did not fire for a script-inserted record.	Called explainViolation from Check Distribution Rights.	Closed
D-07	Dashboard error: Reached end of the page action.	Opening block ended with }%> instead of %>.	Corrected the snippet and added the closing <%}%>.	Closed
D-08	AI text invented advice and wording.	Prompt did not limit allowed advice or vocabulary.	Prompt tightened; remaining wording left as is by decision.	Closed (accepted)
D-09	Custom API option not found in Settings.	It is under Microservices on the Creator home page.	Created it there.	Closed
D-10	Invitation to a test user failed.	E-mail was not yet a user of the Zoho account.	Added the user to the account first, then shared the app.	Closed
D-11	Records lost after re-import.	A .ds export holds structure only.	Re-imported sample data from CSV.	Closed

Bug ID	Description	Root Cause	Fix Applied	Status
D-12	Content Owner could still see other owners' records.	Same cause as D-03; My Content, My Ownership Claims and My Rights Holders reports were advised but not confirmed.	Pending confirmation.	Open

6.5 User Acceptance Testing

Formal user acceptance testing with external users was not carried out. Acceptance was checked role by role by the developer using separate logins (Table 6.1): the Distributor and Finance Manager roles passed after the fixes in Table 6.2, and the Content Owner role passed its functional checks, with the visibility gap D-12 still to be closed.

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

MediaVault was developed and tested in a single application instance; separate development, stage and production environments were not used. Deployment amounted to keeping the application recoverable: the application was exported as a .ds file, re-imported once after the trial lapsed, and the sample data re-loaded from CSV files. Table 7.1 records the checklist as it stands.

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Done (✓)
1	Application exported as a .ds file and re-imported after the trial lapsed	✓
2	Sample data re-loaded from CSV files	✓
3	Test users invited with Content Owner, Distributor and Finance Manager profiles	✓
4	Role tests run with separate logins	✓
5	Custom API created again in the new account and protected with OAuth 2.0	✓
6	Publish through development, stage and production environments	—
7	Data Sharing rules for record visibility	—
8	Content Owner "My" reports confirmed	—

7.2 Performance Optimisation

No performance measurements were taken, so no before-and-after figures are reported. Table 7.2 lists the design choices that keep the data load small; their effect was not measured.

Table 7.2: Performance-Related Design Choices

Design Choice	Component	Measured?	Result	Remarks
Dashboard tiles use count() on criteria instead of fetching and looping over records	DS-10	No	Not available	Six counts per page load
One violation per request is updated instead of inserting duplicates	DS-01	No	Not available	Keeps the violations form small
Criteria-based fetch of licences by content and territory	DS-01	No	Not available	Only matching licences are looped

7.3 Security Features

Table 7.3: Security Controls Implemented

Security Control	Where Applied	Threat Mitigated
Permission profiles without Delete	All forms and reports	Unauthorised viewing, editing or deletion
Filtered My Requests report in place of the all-records report	Distributor profile	A distributor seeing other distributors' requests
Administrator-only verification (zoho.loginuser equals zoho.adminuser) and lock on verified rows	DS-03, DS-04	An owner verifying or altering their own claim
Saved Severity and Request Status restored on edit; fields hidden	Request and violation forms	Users choosing their own severity or approval
Active licence requires Verified ownership	DS-06	Licensing content whose owner is unconfirmed
OAuth 2.0 with a scoped token on the Custom API	Fetch_License_Records	Anonymous reading of licences (shown by the 401 test)
AI prompt limited to facts; output labelled and never used for decisions	DS-09	Leaking personal data; AI changing a decision

Known gaps: the API returns every licence to any holder of a valid token; the Severity field is marked private and it was not checked that authorised roles see it read-only; and no Data Sharing rules were defined.

7.4 Governance and Administration

Governance rests on the role hierarchy, the profiles and the stored evidence in the records themselves: each violation keeps its reason, each verified claim keeps the verifier and date, and records keep the Added User. The application owner is the only administrator. The Creator audit trail and administration settings were not reviewed as part of this project. The trial period limits how long the application remains available, so a fresh .ds export, together with the CSV files, is the recovery path.

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

The first end-to-end result was the rights check for REQ-003, a request for Monsoon Letters in Australia for 01-Nov-2026 to 31-Dec-2026, for which no licence existed. The system set the status and created the violation without the user choosing a severity (Table 8.1).

Table 8.1: Observed Output of the First Rights Check (REQ-003)

Field	Value
Reason	No license exists for this content in the requested territory (Australia) for 01-Nov-2026 to 31-Dec-2026.
Severity	Critical (assigned by the system)
Violation Status	Open
Notes	CRITICAL: No license covers this territory. Escalate to the rights/legal team before any release.

The ownership flow produced a Pending claim for Monsoon Letters (Meridian Pictures, 60.00) that only the administrator could verify; once the claims for a content item totalled 100 per cent and were verified, its Ownership Status became Verified and its licence could be saved as Active. For a High violation (Monsoon Letters, India, 01-Dec-2027 to 31-Jan-2028) the Zia task produced a summary naming licence LIC-001, valid until 31-Dec-2027. The distributor role saw one request (REQ-D01) through My Requests, and the Finance Manager role could enter revenue and royalty payments but only read content and royalty rules.

8.2 Achievement of Objectives

Table 8.2: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
Objective 1 – validate requests against licences	DS-01; Section 4.4	Critical result observed for REQ-003; other scenarios prepared, outcomes not recorded
Objective 2 – automatic severity; no user edits	DS-01; Protect workflows; hidden fields	Severity set by the system in TC-01 and TC-07
Objective 3 – one violation per request with reason	DS-01	Reason text observed in Table 8.1; one-per-request logic in the script
Objective 4 – ownership claims verified by the administrator, 100% cap	DS-03 to DS-05	TC-02 to TC-05
Objective 5 – Active licence needs Verified ownership	DS-06	Block message observed (TC-04)
Objective 6 – role separation	Profiles, roles, My Requests	Distributor and Finance Manager verified; Content Owner partly (D-12 open)
Objective 7 – API, AI explanation, dashboard	DS-08, DS-09, DS-10	401 test observed; explanation text observed; tiles reported OK

8.3 Comparison with the Existing System

Table 8.3: Existing System versus Proposed Application

Parameter	Existing System	Proposed Application	Improvement
Licence dates in the check	Not compared	Start and end dates must cover the request	Expired licences no longer approve requests
Severity	Always High; keyword guess in a separate workflow	Critical, High, Medium or Low from the result	Severity reflects the cause
Who sets status and severity	Users could edit them	System sets them; values restored on edit	Users cannot approve their own request
Ownership	Typed into a subform, unverified	Pending until the administrator verifies; 100% cap	Licences need verified owners
Access	Default Read and Write profiles	Three role profiles and filtered reports	Roles see only their own work
Outside access to licences	Key-based function	OAuth-protected API with computed Coverage_State	No anonymous reading

8.4 Limitations of the Low-Code Approach

- The platform's behaviour had to be discovered by testing. Record visibility did not follow the role hierarchy as expected, which took filtered reports to correct.
- Deluge could not be run outside Zoho, so each script was tested by entering records and reading the error line reported by the platform.
- A .ds export carries structure only; records, invited users and API settings must be rebuilt, which matters when a trial lapses.
- The AI task cannot guarantee wording even with a strict prompt; its use is therefore limited to labelled, advisory text.
- Plan limits (trial duration, AI-call and external-call quotas) are outside the developer's control.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

Most of MediaVault sits at the no-code end: forms, lookups, reports, profiles and roles are configured, not coded. The rules that make it trustworthy sit at the low-code level, in short Deluge scripts that run on every submission. No pro-code component was

needed. The project also showed where configuration alone is not enough: the visibility problem was solved by adding a filtered report, not by code, and the AI feature needed prompt engineering rather than programming. A pro-code layer would become worthwhile for the planned royalty automation and for an API that lets a distributor's platform ask whether a request would be approved.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

MediaVault turns a prototype that approved requests on territory alone into an application that checks licences by content, territory, status and dates, classifies each violation with a system-assigned severity and a reason, and refuses to activate a licence for content whose ownership has not been verified. Access is separated by role, a protected API publishes licence status, a Zia task explains violations in plain English without influencing them, and a dashboard shows the main indicators.

The report is explicit about what remains unproven: the outcomes of most Stage 1 request scenarios were not recorded, the Content Owner visibility fix is unconfirmed, and royalty automation was not built. The tests that were run, the defects they exposed and the fixes applied are recorded in Chapter 6.

9.2 Future Enhancements

- Royalty automation: generate each rights holder's payment from revenue, licence rule and verified ownership share, with rounding rules, duplicate prevention and controlled Pending → Calculated → Paid transitions.
- Close the Content Owner visibility gap with My Content, My Ownership Claims and My Rights Holders reports, and add an owner-specific royalty view.
- Licensee matching, so that a licence belongs to one distributor.
- Record the approving licence on each request through an Approved License lookup.
- Per-role dashboards and a violations-by-severity chart.
- A Check_Rights API that lets a distributor's platform ask whether a request would be approved, with its own authentication.
- E-mail notification of violations was considered and deliberately not pursued; it can be revisited, with rules deciding who is told and AI only wording the message.

9.3 Learning Outcomes and Career Relevance

The project gave practice in modelling a rights domain with lookups, writing server-side validation and classification in Deluge, configuring profiles, roles and filtered

reports, protecting an API with OAuth 2.0, and using AI as an explainer rather than a decision-maker. It also taught habits that matter in professional low-code work: check field link names before scripting, test each role with its own login, read the platform's error line, record exactly what was and was not verified, and keep a recoverable export. These are relevant to roles in low-code development, business-application support and integration.

REFERENCES

[1] A. Bock and U. Frank, "Low-Code Platform," *Business & Information Systems Engineering*, vol. 63, no. 6, pp. 733–740, 2021.

[2] A. Sahay, A. Indamutsa, D. Di Ruscio and A. Pierantonio, "Supporting the understanding and comparison of low-code development platforms," in *Proc. Euromicro Conf. on Software Engineering and Advanced Applications (SEAA)*, 2020, doi: 10.1109/SEAA51224.2020.00036.

[3] Zoho Corporation, "Creating and Managing Custom APIs," Zoho Creator Help. [Online]. Available: https://help.zoho.com/portal/en/kb/creator/developer-guide/microservices/custom-api/articles/create-and-manage-custom-apis (Accessed: 30 October 2026).

[4] Zoho Corporation, "Zia Task," Zoho Deluge Help. [Online]. Available: https://www.zoho.com/deluge/help/ai-tasks/zia-task.html (Accessed: 30 October 2026).

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
I	Application created & sections/menu organised	Y	MediaVault; menu of nine forms	5.1	Fig. 5.1
II	Forms with ≥ 8 distinct field types	N	Fewer field types were used; no field-type audit was done	4.3	Table 4.2
II	Lookup field (relationship between forms)	Y	Content_Ownership, Licenses, Distribution_Requests and others	4.2	Fig. 4.2; Table 4.1
II	Subform (parent–child data)	Y	Legacy Rights_Ownership subform in Content (superseded by Content_Ownership)	1.2, 4.2	–
II	Field validations / layout rules	Y	Content_Ownership; request and licence dates	5.5	DS-04, DS-07
II	List report with filter, grouping, custom action	Y	My Requests; Pending Ownership Claims (filters; no custom action)	5.3	Table 5.1; Fig. 5.5
II	Kanban / Calendar / Timeline / Map report	N	Not used	5.3	–
II	Pivot table / chart report	Y	Revenue by Platform chart on the dashboard	5.4	–
II	Page / dashboard built with UI Builder	Y	MediaVault Dashboard with KPI tile snippet	5.4	DS-10
II	Deluge – form events (validate / success / field actions)	Y	Content_Ownership, Licenses, Distribution_Requests	5.5	Code 5.1; DS-03 to DS-07
II	Deluge – functions, fetch, loops, collections	Y	Fetch_License_Records, explainViolation; loops in DS-04, DS-05	5.5	Appendix C
II	Email / in-app notifications	N	Considered and not pursued	9.2	–
II	Form workflow	Y	Table 5.3	5.6	Fig. 5.6, 5.7

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
II	Scheduled workflow	N	Not used – designed to work without schedules	5.6	–
II	Approval process	N	Not used	5.6	–
II	Blueprint	N	Not used	5.6	–
II	Batch / report workflow	N	Not used	5.6	–
II	Testing & debugging evidence	Y	Chapter 6 test cases and defect log	6	Table 6.1, 6.2
III	Integration with a Zoho service	Y	Zia task	5.9	DS-09
III	Connection / REST API call (invokeurl)	N	Not used	5.7	–
III	Webhook or Creator REST API	Y	Custom API Fetch_License_Records (OAuth 2.0)	5.7	Fig. 5.8, 5.9; DS-08
III	Users, roles and permission profiles	Y	Three profiles; CEO role hierarchy	5.8	Fig. 5.10–5.13
III	Record-level access criteria / customer portal	Y	My Requests (Added User filter); portal not used	5.3, 5.8	Fig. 5.5
III	Zia / AI feature	Y	Violation explanation	5.9	DS-09
III	Analytics (charts / Zoho Analytics)	Y	KPI tiles; Revenue by Platform chart	5.4	DS-10
IV	Performance optimisation applied & measured	N	Design choices only; not measured	7.2	Table 7.2
IV	Deployment across environments	N	Single instance; export and re-import only	7.1	Table 7.1
IV	Security controls (encryption, OAuth, permissions)	Y	Profiles, filtered reports, OAuth 2.0 API	7.3	Table 7.3
IV	Governance (audit trail, administration)	N	Not reviewed	7.4	–
V	Pro-code extension (Catalyst / widgets / custom functions)	N	Not used (custom Deluge functions only)	5.10	–
V	Low-code limitations & spectrum analysis	Y	Sections 8.4 and 8.5	8.4, 8.5	–
—	Mobile app / portal access	N	Not used	5.11	–

Total features implemented: 19 / 32

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain the principles, architecture, and business value of low-code application development using Zoho Creator.	Ch. 1, 2	3	
CO2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, 5 (5.1–5.6), 6	3	
CO3	Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security.	Ch. 4.6, 5.7–5.9	3	
CO4	Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices.	Ch. 7	3	
CO5	Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer.	Ch. 5.9, 5.10, 8.4, 8.5, 9.3	3	

APPENDIX C – COMPLETE DELUGE SOURCE CODE

Listings DS-02 to DS-10 are the scripts as written during the project.

C.1: Distribution_Requests – Rights check (function form) – DS-01

```
// Reference implementation of the rights check (function form)
void validateRequest(int requestID)
{
    expiryWarningDays = 30;
    req = Distribution Requests[ID == requestID];
    if(req.count() == 0)
    {
        return;
    }
    contentID = req.Content.toLong();
    terr = req.Requested_Territory;
    reqStart = req.Requested Start Date;
    reqEnd = req.Requested_End_Date;
    todayDate = zoho.currentdate;
    if(reqStart == null || reqEnd == null || reqEnd < reqStart)
    {
        req.Request_Status = "Pending";
        req.Remarks = "Cannot validate: requested dates are missing or
the
            end date is before the start date. Administrator review
            needed.";
        return;
    }
    licenses = Licenses[Content == contentID && (Territory == terr ||
        Territory == "Worldwide")];
    bestID = 0;
    bestEnd = todayDate;
    reasons = List();
    dataIssue = false;
    for each  lic in licenses
    {
        lid = lic.License_ID;
        if(lic.License_Start_Date == null || lic.License_End_Date ==
null
            || lic.License_End_Date < lic.License_Start_Date)
        {
            dataIssue = true;
            reasons.add(lid + " has missing or inconsistent dates");
        }
        else if(lic.License_Status != "Active")
        {
            reasons.add(lid + " has status " + lic.License_Status + ",
not
                Active");
        }
        else if(lic.License_Start_Date <= reqStart &&
lic.License End Date
            >= reqEnd)
        {
            if(bestID == 0 || lic.License_End_Date > bestEnd ||
                (lic.License_End_Date == bestEnd && lic.ID < bestID))
            {
                bestID = lic.ID;
                bestEnd = lic.License_End_Date;
            }
        }
```

```
        else if(lic.License_End_Date < todayDate)
        {
            reasons.add(lid + " expired on " +
                lic.License_End_Date.toString("dd-MMM-yyyy"));
        }
        else
        {
            reasons.add(lid + " covers " +
                lic.License_Start_Date.toString("dd-MMM-yyyy") + " to
" +
                lic.License End Date.toString("dd-MMM-yyyy") + " only,
not
                the requested period");
        }
    }
    reqStatus = "";
    remarks = "";
    severity = "";
    vioReason = "";
    vioStatus = "Open";
    violationNeeded = false;
    if(bestID != 0)
    {
        reqStatus = "Approved";
        bestRec = Licenses[ID == bestID];
        remarks = "Approved under license " + bestRec.License_ID + ",
            valid until " + bestEnd.toString("dd-MMM-yyyy") + ".";
        if(bestEnd <= reqEnd.addDay(expiryWarningDays))
        {
            violationNeeded = true;
            severity = "Medium";
            vioReason = "Approved, but license " + bestRec.License_ID
+ "
                ends on " + bestEnd.toString("dd-MMM-yyyy") + ",
within "
                + expiryWarningDays + " days of the requested end
date.
                Renewal needs attention.";
        }
        else
        {
            contentRec = Content[ID == contentID];
            total = 0.0;
            for each  row in contentRec.Rights_Ownership
            {
                if(row.Ownership_Percentage != null)
                {
                    total = total +
row.Ownership_Percentage.toDecimal();
                }
            }
            if((total - 100).abs() > 0.005)
            {
                violationNeeded = true;
                severity = "Low";
                vioReason = "Administrative gap: ownership percentages
for
                    this content total " + total + "%, not 100%. Part
of
                    the royalty cannot be allocated.";
            }
        }
    }
```

```
    else if(dataIssue)
    {
        reqStatus = "Pending";
        remarks = "Cannot decide: " + reasons.toString("; ") + ". Sent
to
            administrator review.";
        violationNeeded = true;
        severity = "";
        vioStatus = "Under Review";
        vioReason = remarks;
    }
    else if(licenses.count() == 0)
    {
        reqStatus = "Rights Violation";
        violationNeeded = true;
        severity = "Critical";
        vioReason = "No license exists for this content in " + terr +
".";
        remarks = vioReason;
    }
    else
    {
        reqStatus = "Rights Violation";
        violationNeeded = true;
        severity = "High";
        vioReason = "A license exists for " + terr + " but it cannot
cover
            this request: " + reasons.toString("; ") + ".";
        remarks = vioReason;
    }
    req.Request_Status = reqStatus;
    req.Remarks = remarks;
    existing = Rights Violations[Distribution Requests == requestID];
    if(violationNeeded)
    {
        if(existing.count() > 0)
        {
            existing.Reason = vioReason;
            existing.Severity = severity;
            existing.Territory = terr;
        }
        else
        {
            insert into Rights_Violations
            [
                Added_User = zoho.loginuser
                Violation_ID = "VIO-" + req.Request_ID
                Distribution_Requests = requestID
                Content = contentID
                Territory = terr
                Violation_Date = todayDate
                Reason = vioReason
                Severity = severity
                Violation_Status = vioStatus
                Notes = "Automatically classified by MediaVault."
            ];
        }
    }
    else
    {
        stillOpen = Rights_Violations[Distribution_Requests ==
requestID
```

```
            && (Violation_Status == "Open" || Violation_Status ==
"Under
            Review")];
        if(stillOpen.count() > 0)
        {
            stillOpen.Violation_Status = "Resolved";
            stillOpen.Notes = "Auto-resolved on re-check: a valid
license
                now covers this request.";
        }
    }
}
```

C.2: Distribution_Requests – Call to explainViolation – DS-02

```
// Script ID : DS-02
// Location  : Distribution_Requests -> Check Distribution Rights (end
of
    script)
for each  v in Rights_Violations[Distribution_Requests == input.ID]
{
    thisapp.explainViolation(v.ID);
}
```

C.3: Content_Ownership – On Load – DS-03

```
// Script ID : DS-03
// Location  : Content_Ownership -> On Load (Created or Edited)
hide Verified_By;
hide Verification_Date;
if(zoho.loginuser != zoho.adminuser)
{
    disable Verification_Status;
}
```

C.4: Content_Ownership – On Validate – DS-04

```
// Script ID : DS-04
// Location  : Content_Ownership -> Validations on form submission
isAdmin = (zoho.loginuser == zoho.adminuser);
errorMsg = "";
savedStatus = "Pending";
if(input.ID != null)
{
    saved = Content_Ownership[ID == input.ID];
    savedStatus = saved.Verification_Status;
}
if(input.Ownership_Percentage == null || input.Ownership_Percentage <=
0
    || input.Ownership_Percentage > 100)
{
    errorMsg = "Ownership percentage must be between 0.01 and 100.";
}
// Only the administrator can change the status
if(errorMsg == "" && !isAdmin)
{
    if(input.ID == null)
    {
        input.Verification Status = "Pending";
```

```
    }
    else if(savedStatus == "Verified")
    {
        errorMsg = "Verified ownership is locked. Ask an administrator
to
            change it.";
    }
    else
    {
        input.Verification_Status = savedStatus;
    }
}
// One claim per holder per content
if(errorMsg == "")
{
    for each  p in Content_Ownership[Content == input.Content &&
        Rights_Holders == input.Rights_Holders && Verification_Status
!=
        "Rejected"]
    {
        if(input.ID == null || p.ID != input.ID)
        {
            errorMsg = "This rights holder already has an ownership
claim
                for this content.";
        }
    }
}
// Verified shares for one content can never exceed 100%
if(errorMsg == "" && input.Verification_Status == "Verified")
{
    total = input.Ownership_Percentage;
    for each  p in Content Ownership[Content == input.Content &&
        Verification_Status == "Verified"]
    {
        if(input.ID == null || p.ID != input.ID)
        {
            total = total + p.Ownership_Percentage;
        }
    }
    if(total > 100.0001)
    {
        errorMsg = "Verified ownership for this content would total "
+
            total + "%. It cannot exceed 100%.";
    }
}
if(errorMsg != "")
{
    alert errorMsg;
    cancel submit;
}
else
{
    if(input.Verification_Status == "Verified" && savedStatus !=
        "Verified")
    {
        input.Verified_By = zoho.loginuser;
        input.Verification_Date = zoho.currentdate;
    }
    else if(input.Verification_Status != "Verified")
    {
        input.Verified_By = "";
```

```
        input.Verification_Date = null;
    }
}
```

C.5: Content_Ownership – On Success – DS-05

```
// Script ID : DS-05
// Location  : Content Ownership -> Update Content Ownership Status
(On
    Success)
verifiedTotal = 0.0;
pendingCount = 0;
rowCount = 0;
for each  p in Content_Ownership[Content == input.Content]
{
    rowCount = rowCount + 1;
    if(p.Verification_Status == "Verified")
    {
        verifiedTotal = verifiedTotal + p.Ownership_Percentage;
    }
    else if(p.Verification_Status == "Pending")
    {
        pendingCount = pendingCount + 1;
    }
}
newStatus = "Pending";
if(verifiedTotal >= 99.995 && verifiedTotal <= 100.005)
{
    newStatus = "Verified";
}
else if(verifiedTotal > 0)
{
    newStatus = "Partially Verified";
}
else if(rowCount > 0 && pendingCount == 0)
{
    newStatus = "Rejected";
}
c = Content[ID == input.Content];
c.Ownership Status = newStatus;
```

C.6: Licenses – On Validate – DS-06

```
// Script ID : DS-06
// Location  : Licenses -> Validations on form submission
if(input.License Status == "Active")
{
    c = Content[ID == input.Content];
    if(c.Ownership_Status != "Verified")
    {
        alert "Cannot activate this license: ownership of this content
is
            not verified yet.";
        cancel submit;
    }
}
```

C.7: Distribution_Requests – On Validate – DS-07

```
// Script ID : DS-07
// Location  : Distribution_Requests -> On Validate (Add and Edit)
if(input.Requested_End_Date < input.Requested_Start_Date)
{
    alert "Requested end date must be on or after the start date.";
    cancel submit;
}
```

C.8: Function Fetch_License_Records – DS-08

```
// Script ID : DS-08
// Location  : Functions -> Fetch_License_Records (behind the Custom
API)
map Fetch_License_Records()
{
    todayDate = zoho.currentdate;
    result = List();
    for each  lic in Licenses[ID != 0]
    {
        state = "Not Active (" + lic.License_Status + ")";
        if(lic.License Status == "Active")
        {
            if(lic.License End Date != null && lic.License End Date <
                todayDate)
            {
                state = "Expired";
            }
            else if(lic.License_Start_Date != null &&
                lic.License_Start_Date > todayDate)
            {
                state = "Not Started";
            }
            else if(lic.License_End_Date != null &&
lic.License_End_Date
                <= todayDate.addDay(30))
            {
                state = "Expiring Soon";
            }
            else
            {
                state = "Valid";
            }
        }
        endText = "";
        if(lic.License_End_Date != null)
        {
            endText = lic.License_End_Date.toString("dd-MMM-yyyy");
        }
        item = Map();
        item.put("License_ID",lic.License_ID);
        item.put("Licensee_Name",lic.Licensee_Name);
        item.put("Territory",lic.Territory);
        item.put("License Status",lic.License Status);
        item.put("License_End_Date",endText);
        item.put("Coverage State",state);
        result.add(item);
    }
    response = Map();
    response.put("status","success");
    response.put("count",result.size());
    response.put("licenses",result);
```

```
    return response;
}
```

C.9: Function explainViolation – DS-09

```
// Script ID : DS-09
// Location  : Functions -> explainViolation (called from DS-02)
void explainViolation(int violationRecID)
{
    v = Rights_Violations[ID == violationRecID];
    if(v.count() == 0)
    {
        return;
    }
    req = Distribution Requests[ID ==
v.Distribution_Requests.toLong()];
    cont = Content[ID == v.Content.toLong()];
    // Facts only. No emails, phone numbers or personal names.
    facts = "Content: " + cont.Title + ". Territory requested: " +
        v.Territory;
    if(req.count() > 0)
    {
        facts = facts + ". Requested period: " +
            req.Requested_Start_Date.toString("dd-MMM-yyyy") + " to "
+
            req.Requested_End_Date.toString("dd-MMM-yyyy");
    }
    facts = facts + ". Severity: " + v.Severity + ". System reason: "
+
        v.Reason;
    prompt = "You write short notices for a media licensing system.
The
        reader is a distributor who asked for distribution rights.
Using
        ONLY the facts below, write 2 to 3 plain sentences. Sentence
1:
        say what the distributor requested and why it was refused,
using
        only the license ID, territory and dates given. Always call
the
        reader 'the distributor' and the activity 'distribution'.
Never
        use the words licensor, broadcast, stream, release or publish.
        Sentence 2: the next step. The only next steps you may suggest
        are: (a) ask the rights owner to extend or renew the license,
(b)
        request a license for the missing territory, or (c) resubmit
with
        dates inside the licensed period. Do not mention authorities,
        deadlines, urgency, penalties, legal advice or any person or
        organisation not named in the facts. Do not invent license
numbers
        or dates, and do not change the severity. Facts: " + facts;
    try
    {
        response = Zia
        [
            message : prompt
        ];
        v.AI_Explanation = "AI-generated summary. The stored Reason
and
            Severity are the official result. " + response.toString();
```

```
    }
    catch (e)
    {
        v.AI_Explanation = "AI explanation unavailable: " + e;
    }
}
```

C.10: Dashboard KPI tiles (HTML snippet with Deluge) – DS-10

```
<%{
openViol = Rights_Violations[Violation_Status == "Open"].count();
critical = Rights_Violations[Severity == "Critical" &&
Violation_Status ==
    "Open"].count();
pendingClaims = Content Ownership[Verification Status ==
    "Pending"].count();
activeLic = Licenses[License_Status == "Active" && License_End_Date >=
    zoho.currentdate].count();
expiring = Licenses[License_Status == "Active" && License_End_Date >=
    zoho.currentdate && License_End_Date <=
    zoho.currentdate.addDay(30)].count();
pendingReq = Distribution_Requests[Request_Status ==
"Pending"].count();
%>
<style>
.mv-grid{display:grid;grid-template-columns:repeat(auto-
fit,minmax(160px,1fr));gap:14px;padding:8px}
.mv-card{background:#fff;border:1px solid #e6e8ee;border-left:5px
solid
    #4f46e5;border-radius:12px;padding:16px 18px;font-family:sans-
serif}
.mv-card b{display:block;font-size:32px;color:#1b1f3b}
.mv-card span{font-size:13px;color:#6b7280}
.mv-red{border-left-color:#dc2626}.mv-amber{border-left-
color:#f59e0b}.mv-green{border-left-color:#16a34a}
</style>
<div class="mv-grid">
<div class="mv-card mv-red"><b><%=openViol%></b><span>Open
    violations</span></div>
<div class="mv-card mv-red"><b><%=critical%></b><span>Critical
    (open)</span></div>
<div class="mv-card mv-amber"><b><%=pendingClaims%></b><span>Ownership
    claims pending</span></div>
<div class="mv-card mv-green"><b><%=activeLic%></b><span>Active
    licenses</span></div>
<div class="mv-card mv-amber"><b><%=expiring%></b><span>Expiring
within 30
    days</span></div>
<div class="mv-card"><b><%=pendingReq%></b><span>Requests
    pending</span></div>
</div>
<%}%>
```

APPENDIX D – USER MANUAL

D.1 Accessing the Application

Open the MediaVault link in a browser and sign in with the Zoho account provided by the Administrator. Each role sees only its own menu. Test users must first be added to the Zoho account and then shared the application with the matching permission and role.

D.2 Instructions for the Administrator

- Open Pending Ownership Claims, open a claim and set Verification Status to Verified or Rejected; Verified By and Date are filled in automatically.
- When all claims of a content item total 100 per cent and are Verified, its Ownership Status becomes Verified and licences for it can be saved as Active.
- Add licences with content, licensee, territory, dates and status.
- Open Rights Violations to read each violation's severity, reason and AI explanation; the stored Reason and Severity are the official result.
- Open the MediaVault Dashboard to read the KPI tiles.

D.3 Instructions for the Content Owner

- Add a content record, then add an ownership claim for it in Content_Ownership with the rights holder and percentage.
- The claim is saved as Pending; you cannot change its status or edit it after saving.

D.4 Instructions for the Distributor

- Open Distribution Requests, fill in request ID, content, platform, territory, dates and expected revenue, and submit.
- Open My Requests to see your requests; Request Status and Remarks are set by the system.

D.5 Instructions for the Finance Manager

- Open Revenue to add revenue for a content item and licence, and Royalty Payments to add a payment.
- A duplicate rights holder for the same revenue is blocked, and a payment marked Paid cannot be changed.

APPENDIX E – EVALUATION RUBRIC

(For Examiner Use Only)

Student Name	PRAGADEESH S	DATE	2116240701388
Project Title	MEDIAVAULT – DIGITAL MEDIA RIGHTS, LICENSING, DISTRIBUTION & ROYALTY MANAGEMENT SYSTEM	Date:	

Table E.1: Evaluation Rubric for the Individual Project

S.No.	**Criterion**	**Indicators**	**Max. Marks**	**Marks Awarded**
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		100	

Table E.2: Performance Level Descriptors

Level	% of Max. Marks	Descriptor
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Signature of Internal Examiner	**Signature of External Examiner**
Name: _____	Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

(For Examiner Use Only)

Student Name	**PRAGADEESH S**	**DATE**	2116240701388
Department / Section	CSE – G	Date of Viva:	
Application URL Verified	Yes / No	Demo Duration	_____ min

Table F.1: Viva-Voce Questions and Responses

Q.No.	**Question / Topic Asked**	**Unit**	**Response (E / G / S / NI)**	**Remarks**
1				
2				
3				
4				
5				
6				
7				
8				

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Table F.2: Marks Summary

Component	**Max. Marks**	**Marks Awarded**
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	100	

Examiners' Remarks:

_____

_____

INTERNAL EXAMINER	**EXTERNAL EXAMINER**
Name: _____	Name: _____